🎯 BA Interview Preparation — Part 1: Basic Questions

1. Tell me about yourself / Walk me through your profile

What interviewer wants

They want a 60–90 second summary, not your entire life story.

Your answer should cover:

Education → Previous experience → Career transition → BA skills → Projects → Target role

Interview-ready answer

I have a background in pharmacy, with a Master's degree in Pharmacology, and I started my professional career in the pharmaceutical industry.

I have experience in regulatory affairs and business development, where I worked with business requirements, documentation, coordination with different stakeholders, and process-related activities.

Over time, I developed an interest in technology and decided to transition into Business Analysis. To build practical IT BA skills, I have worked on hands-on healthcare projects covering areas such as Hospital Management, Clinical Trial Management, Healthcare Revenue Cycle Management, and Healthcare Analytics.

As part of these projects, I have practiced requirement gathering, stakeholder analysis, BRD and FRD documentation, process modelling, BPMN/UML, user stories, acceptance criteria, backlog management, Jira, SQL, API concepts, UAT, and Agile/Scrum practices.

My domain background in pharmaceuticals and healthcare gives me an additional advantage when working on healthcare technology products.

Now I am looking for a Business Analyst opportunity where I can combine my healthcare domain knowledge with my BA and technology skills and contribute to real-world product and project delivery.

Tip: Don't say "I don't have IT experience" in the opening. Let your background and practical work establish the transition naturally. If they specifically ask about real production experience, answer honestly.

---

2. What are your roles & responsibilities as a BA?

A strong answer:

As a Business Analyst, my primary responsibility is to understand the business problem and translate it into clear and actionable requirements for the development and QA teams.

My responsibilities include:

1. Understanding business objectives and problems.
2. Identifying and communicating with stakeholders.
3. Gathering requirements through interviews, meetings, workshops and document analysis.
4. Analyzing and validating requirements.
5. Preparing BRD, FRD and other requirement documentation.
6. Creating process flows, use cases, user journeys and UML/BPMN diagrams where required.
7. Creating user stories and acceptance criteria in Agile projects.
8. Working with Product Owners and developers during backlog refinement and sprint planning.
9. Clarifying requirements during development.
10. Supporting QA by explaining business scenarios and validating test cases.
11. Supporting UAT and coordinating business feedback.
12. Supporting deployment and post-production validation.
13. Managing requirement changes and maintaining traceability.

Remember this formula:

Business Problem → Requirements → Documentation → Development → Testing → UAT → Deployment → Feedback

That is the BA lifecycle you should keep in your head.

---

3. Tell me about your day-to-day activities. How do you start your day?

Don't say:

"I attend standup and check emails."

That's too weak.

Say:

I usually start my day by checking my emails, Teams or project communication channels for any new business updates, issues or dependencies.

Then I review my Jira board and check the status of the stories I am working on. If there is a daily Scrum, I attend the stand-up and understand what the development and QA teams are working on and whether there are any blockers related to requirements.

After that, my activities depend on the project phase. I may conduct requirement discussions with stakeholders, clarify questions from developers or testers, update user stories and acceptance criteria, review test scenarios, participate in backlog refinement, or support UAT.

I also make sure that any important decisions, requirement changes or stakeholder feedback are properly documented and communicated to the relevant team members.

Very important

If they ask:

"What happens after stand-up?"

You should be able to explain your actual BA workflow, not just Scrum ceremonies.

A strong interview answer would be:

After the stand-up, I first look at whether there are any blockers, requirement clarifications, or dependencies that need my attention.

For example, if a developer has a question about a user story or acceptance criteria, I connect with the developer and clarify the requirement with the business context. If QA has questions about a test scenario, I help clarify the expected behavior.

Then I prioritize my BA activities for the day. Depending on the project phase, I may work on requirement analysis, update user stories and acceptance criteria, conduct stakeholder discussions, participate in backlog refinement, review test scenarios, support UAT, or analyze defects and change requests.

I also follow up on any action items from previous meetings and make sure important decisions or requirement changes are documented and communicated to the relevant stakeholders.

So, for me, the stand-up is mainly to understand the team's current status, and after the stand-up I focus on the requirement, clarification, documentation, testing, or stakeholder activities that need my attention that day.

If they ask a follow-up: "Give me an example."

Use a concrete example:

For example, suppose during stand-up a developer says, "I have a question about the eligibility verification story because it's not clear what should happen when the payer API does not return a response."

After stand-up, I would review the story and acceptance criteria, understand the exact gap, and discuss it with the developer and QA. If the business rule is not already defined, I would confirm it with the Product Owner or business SME.

For example, we may decide that if the API times out, the system should retry once and then show an appropriate status for manual follow-up.

I would then update the acceptance criteria or relevant requirement documentation, communicate the decision to the team, and make sure QA knows the expected behavior for testing.

---

4. As a BA, how do you gather requirements and prepare for a stakeholder meeting?

Use this framework:

Before → During → After

Before meeting

- Understand business objective
- Review existing documents
- Identify stakeholders
- Prepare questions
- Understand current process
- Identify known issues
- Prepare agenda

During

- Ask open-ended questions
- Understand AS-IS
- Understand pain points
- Understand desired TO-BE
- Capture functional requirements
- Capture NFRs
- Identify assumptions/dependencies
- Confirm priorities

After

- Prepare MoM
- Document requirements
- Validate understanding
- Share for stakeholder confirmation
- Update BRD/FRD/user stories

Interview answer

I normally prepare for a stakeholder meeting in three stages.

Before the meeting, I understand the business objective, review existing documentation and identify the relevant stakeholders. I prepare an agenda and a list of questions based on the current process and known pain points.

During the meeting, I first understand the AS-IS process, identify the problems or gaps, and then understand the desired TO-BE process. I ask questions around business rules, users, exceptions, integrations, data, reporting and non-functional requirements. I also confirm priorities and dependencies.

After the meeting, I document the discussion, prepare the requirements or update the relevant documentation, capture action items and open questions, and share the understanding with stakeholders for validation.

---

5. What is your role as a BA in Testing?

This is a very important interview question.

BA does not normally execute all testing.

Your role is primarily to ensure that testing validates the business requirements.

BA involvement

Requirement → Acceptance Criteria → Test Scenarios → Test Cases → UAT → Defects → Business Validation

Answer

As a BA, I am involved throughout the testing lifecycle, although I am not the primary person responsible for executing technical testing.

I make sure that requirements and acceptance criteria are clear and testable. During test preparation, I work with QA to clarify business scenarios, business rules and expected outcomes.

I review test scenarios from a business perspective and make sure important positive, negative and exception scenarios are covered.

During defect discussions, I help determine whether the observed behavior is actually a defect or an expected business behavior.

During UAT, I support business users, clarify requirements, help validate the results and collect feedback.

So my main responsibility in testing is to make sure that the developed solution behaves according to the approved business requirements.

---

6. What is your role as a BA in Deployment?

A BA is not normally responsible for technically deploying the application.

DevOps/development/technical teams generally handle deployment.

BA supports the business readiness and validation.

Answer

As a BA, I don't normally perform the technical deployment myself. The development or DevOps team handles that activity.

My responsibility is to support business readiness for deployment.

Before deployment, I verify that the required functionality has passed testing and UAT and that the business stakeholders are aware of the changes.

I may support release notes, business communication, training material or process updates.

After deployment, I participate in smoke or business validation testing, monitor whether the expected functionality is working correctly and collect feedback from business users.

If an issue occurs, I help determine the business impact and coordinate with the appropriate technical team.

📌 **PROJECT QUESTIONS**

For your project questions, we should use one consistent project story.

For example:

Healthcare RCM Platform

Problem:

Hospitals have difficulty tracking claims, denials, payments and outstanding receivables.

Users:

- **Billing staff**
- **Revenue cycle managers**
- **Finance team**
- **Hospital administrators**

Workflow:

Patient registration
↓
Insurance verification
↓
Charge capture
↓
Claim creation
↓

Claim submission
↓
Payer response
↓
Payment posting
↓
Denial management
↓
AR follow-up
↓
Reporting

This single project can help you answer 20+ interview questions.

---

7. Explain the project workflow & user journey

Answer structure

User → Action → System → Outcome

Example:

In my healthcare RCM project, the user journey starts when a patient receives a healthcare service.

First, the patient's demographic and insurance information is captured. The system then verifies the patient's insurance eligibility.

Once the service is completed, the required charges and clinical information are captured and a claim is generated.

The claim is then submitted to the payer through an integration. The system receives the payer response and updates the claim status.

If the claim is paid, the payment is posted to the patient's account. If the claim is rejected or denied, the denial is categorized and assigned for follow-up.

Finally, the revenue cycle manager can use dashboards and reports to monitor claim status, denial trends, outstanding receivables and collection performance.

From a BA perspective, I mapped this journey to identify business processes, user roles, system interactions, exceptions and reporting requirements.

---

8. What was your contribution as a BA?

Don't say:

"I created BRD and user stories."

That's too generic.

Show the complete BA lifecycle.

My contribution covered the project from requirement discovery through UAT.

I started by understanding the business problem and identifying the key stakeholders and user roles.

I analyzed the AS-IS process and designed the TO-BE process. I documented the requirements and created process flows and user journeys.

For Agile delivery, I converted the requirements into epics, features and user stories with acceptance criteria.

During development, I worked as the bridge between the business and technical teams and clarified requirement-related questions.

I also supported QA by reviewing business scenarios and helped during UAT by validating whether the implemented functionality met the expected business requirements.

Finally, I supported post-implementation feedback and identified improvement opportunities.

---

9. Project challenges and how did you overcome them?

Use STAR:

Situation → Task → Action → Result

Example:

Challenge: Changing requirements

One challenge I faced was changing requirements during the development phase.

For example, a stakeholder initially wanted a basic denial report, but later requested additional filtering by payer, denial reason and date range.

Instead of directly asking the development team to implement the change, I first understood the business reason behind the request.

I analyzed the impact on existing requirements, UI, database, reporting and development effort. I discussed the impact with the Product Owner and technical team and helped prioritize the change.

If it was critical, we included it in the current sprint only after understanding the impact. Otherwise, we moved it to the product backlog for prioritization.

This helped us avoid uncontrolled scope changes while still addressing the stakeholder's business need.

---

10. APIs integrated into your project

This question needs special preparation because interviewers may go deep.

You need to understand:

API → Request → Endpoint → Authentication → Payload → Response → Error handling → Integration → Business outcome

Interview-ready answer

“In my healthcare RCM portfolio project, I worked through the API integration requirements and designed the integration flow from a Business Analyst perspective.

The main integrations I considered were an **insurance eligibility API, claim submission API, payment or remittance integration, and patient or EMR integration**.

For example, for insurance eligibility, the RCM system sends patient and insurance information to the payer or eligibility service through an API. The request contains information such as member ID, payer details, and patient demographics.

From a BA perspective, I focused on understanding the **endpoint, request and response payload, authentication, required fields, business rules, and error scenarios**.

For example, if the eligibility response says the patient is active, the RCM system can continue with the billing process. If the patient is inactive, the system should flag the case for follow-up before the claim is submitted.

For claim submission, the system sends the required claim information to the appropriate external service. I would define what information is required, how the response should be handled, and what should happen if the claim is rejected, the API times out, or the external system is unavailable.

I also considered asynchronous communication using **webhooks and polling**. For example, if a payer supports webhooks, the payer can notify our system when a claim status changes. If the webhook is unavailable, the system can use polling to periodically check the claim status.

I also considered a fallback mechanism. For example, webhook can be the primary method, and if the webhook fails, polling can be used as the backup.

So, I did not approach the integration as a developer writing the API code. My BA responsibility was to understand the **business purpose of the integration, data flow, requirements, business rules, error handling, dependencies, and expected business outcome**, and make sure these requirements were clear for development and testing."

🔥 **If interviewer asks: "Can you explain one API in detail?"**

Use **Eligibility API** because it is easy to explain in Healthcare RCM.

Flow:

Patient Registration

↓

Insurance Details Captured

↓

Eligibility API Request

↓

Authentication

↓

Payer / Insurance System

↓

API Response

↓

RCM System

↓

Eligible / Ineligible / Error

You can explain:

“Suppose a patient comes to the hospital. Before the service is provided, the RCM system needs to verify whether the patient's insurance is active. The system sends the patient's insurance information to the payer through an eligibility API.”

The interviewer may then ask you these **deep-dive questions**:

Interviewer may ask	**Your answer should cover**
What is the endpoint?	URL where the API operation is exposed
What is the request?	Data sent to the payer
What is the response?	Eligibility information returned
Authentication?	How the calling system proves it is authorized
Payload?	JSON/XML data structure
What if API fails?	Timeout, retry, error status, fallback/manual handling
What if patient is inactive?	Business rule → flag for follow-up
Who owns the API?	Internal/external system or payer
How did you test it?	Postman/API testing + expected responses
What does BA document?	API requirements, field mapping, business rules, errors, dependencies

For healthcare RCM, examples could include:

- **Insurance eligibility API**
- **Claim submission API**
- **Payment API**

- **Patient/EMR integration API**
- **Notification API**

But be careful: don't claim you integrated production APIs if you didn't actually do so.

You can say:

"In my portfolio project, I designed/worked through the integration flow..."

rather than falsely saying:

"I integrated this production API."

⭐ Webhook vs Polling vs Fallback

Interview-ready answer:

“Webhook, polling, and fallback are three different ways of handling communication between systems.

A webhook is an event-based mechanism. Instead of our system continuously asking for an update, the external system sends us a notification when something happens.

For example, in a healthcare RCM system, suppose a claim is submitted to a payer and later the claim status changes. The payer can send a webhook notification to our RCM system saying that the claim status has changed. Our system can then update the claim status.

Polling is different. In polling, our system periodically asks the external system whether there is an update.

Polling is a mechanism where one system repeatedly sends requests to another system to check whether new information or a status change is available.

For example, the RCM system may check the payer system every few minutes:
‘Has the claim status changed?’

If the payer says the claim is approved, rejected, or denied, our system updates the claim accordingly.

A fallback is a backup mechanism. If the primary communication method is unavailable or fails, the system uses another method.

For example, we can use a webhook as the primary method, but if the webhook notification is not received, the system can use polling as a fallback to check the claim status.

So, in simple terms:

Webhook = ‘Tell me when something happens.’

Polling = ‘I will keep checking whether something happened.’

Fallback = ‘If my preferred method fails, I will use another method.’

From a BA perspective, I would define when each mechanism should be used, what data should be exchanged, what happens when communication fails, retry rules, error handling, and the expected business outcome.”

WEBHOOK

Payer → RCM

"Something changed."

↓

Update

POLLING

RCM → Payer

"Anything changed?"

"Anything changed?"

"Anything changed?"

↓

Update

FALLBACK

Primary → Webhook

↓ fails

Backup → Polling

If interviewer asks: “Which one is better?”

Don't say one is always better.

Say:

“It depends on the business and technical requirements. Webhooks are useful when we need event-based and timely updates. Polling is useful when the external system doesn't support webhooks or when we need a controlled periodic check. A fallback mechanism improves reliability when the primary communication method fails.”

One important BA point

The interviewer may try to take you deeper:

“What happens if the webhook fails?”

A good BA answer:

“I would define a retry mechanism or fallback process. For example, if a webhook notification is not successfully delivered, the system could retry based on the agreed retry policy. If it still fails, polling can periodically check the claim status. The requirement should also define logging, error handling, and when the issue should be escalated for manual intervention.”

📌 DOCUMENTATION

11. What is BRD?

Interview-ready answer

“BRD stands for Business Requirements Document.

It is a document that captures the business problem, business objectives, stakeholder needs, scope, and high-level requirements of a project.

As a Business Analyst, I use the BRD to make sure that everyone has a common understanding of what the business wants and why the project is needed, before getting into detailed technical or functional requirements.

For example, in my healthcare RCM portfolio project, the business problem could be that claim processing is taking too much manual effort and causing delays.

The BRD would capture the objective, such as improving claim processing efficiency, reducing manual work, and improving visibility of claim status.

It would also include the project scope, key stakeholders, high-level business requirements, assumptions, constraints, dependencies, and sometimes business rules, depending on the organization's documentation process.

Once the business requirements are agreed upon, they can be further broken down into functional requirements, user stories, acceptance criteria, and technical requirements.

So, in simple terms, I would say:

BRD explains what the business needs, why it needs it, and what the project is expected to achieve."

🧠 *Easy way to remember*

BRD = Business perspective

Problem → Why → Business Objective → Scope → High-level Requirements

If interviewer asks: "Who prepares the BRD?"

You can say:

"Typically, the Business Analyst is responsible for creating and maintaining the BRD, based on information gathered from stakeholders. The relevant business stakeholders review and validate it, and the approval process depends on the organization's governance."

If they ask: "What is included in a BRD?"

Mention the important sections:

1. Author, change control and approval
2. Executive summary

3. Glossary
4. Business goal
5. Project overview and objective
6. Project scope
7. Success criteria
8. Current state-target state
9. RAID (Risk, assumption, issue, dependency)
10. Stakeholders
11. High-level business requirements
12. Business rules
13. Assumptions
14. Constraints
15. Dependencies
16. Out of scope
17. Approval/sign-off

Interview tip: Don't list 20 sections unless asked. Start with the purpose of the BRD, then give the RCM example. That sounds much more natural in an interview.

For interview purposes, I'd say:

“The exact BRD structure varies by organization, but typically it includes the executive summary, business problem and objectives, scope and out-of-scope, stakeholders, current and target state, high-level business requirements, business rules, assumptions, constraints, dependencies, success criteria, and approval. In some organizations, risks, assumptions, issues, and dependencies are maintained together in a RAID section.”

Think of BRD as:

Why → What → Who → Scope → Current/Target → Requirements → Rules → Risks/Dependencies → Success → Approval

For your Healthcare RCM project, for example:

Why: Claim processing has delays and manual work
↓
Goal: Improve claim processing efficiency
↓
Scope: Eligibility, claim submission, denial management
↓
Current State: Manual verification and follow-ups
↓
Target State: Automated eligibility and claim-status workflow
↓
Requirements: High-level business requirements
↓
Business Rules: Eligibility/claim processing rules
↓
Risks/Dependencies: Payer APIs, data quality, external systems
↓
Success: Reduced manual effort and faster processing
↓
Approval: Business stakeholders approve the requirements

That is the BA mindset behind a BRD, rather than just a list of sections.

---

12. BRD vs FRD

Interview-ready answer

“BRD stands for Business Requirements Document, while FRD stands for Functional Requirements Document.

The main difference is the level and perspective of the requirements.

The BRD focuses on the business perspective — it explains the business problem, why the project is needed, business objectives, scope, and what the business wants to achieve.

The FRD focuses on the system functionality — it explains how the system should behave to meet those business requirements. It contains detailed functional requirements, workflows,

validations, business rules, inputs and outputs, error handling, integrations, and other functional details.

For example, in a healthcare RCM project:

BRD requirement:
‘The business wants to automate insurance eligibility verification to reduce manual effort and claim processing delays.’

FRD requirement:
‘The system shall allow an authorized billing user to initiate eligibility verification, send the patient's insurance information to the eligibility service, receive the response, display the eligibility status, and handle inactive coverage, missing information, and API timeout scenarios.’

So, in simple terms:

BRD = What the business needs and why.

FRD = What the system should do to satisfy that business need.

The exact documentation structure can vary between organizations, and some organizations may combine or replace these documents with SRS, functional specifications, or Agile artifacts.”

BRD	FRD
Business Requirements Document	Functional Requirements Document
Business perspective	System/function perspective
WHAT & WHY	HOW the system should behave functionally
High-level	More detailed
Business stakeholders	Business + BA + technical/QA teams
Business objectives	Functional behavior
Business requirements	Functional requirements

Easy example

BRD:

The hospital should be able to track insurance claim status.

FRD:

The system shall display claim status as Submitted, Accepted, Rejected, Denied or Paid.

---

13. Explain FRD in detail

Interview-ready answer

“FRD stands for Functional Requirements Document.

It describes how the system should behave to fulfill the business requirements. While the BRD focuses more on the business need and what the business wants to achieve, the FRD goes into more detail about the system functionality.

As a Business Analyst, I prepare the FRD by analyzing the business requirements and converting them into detailed functional requirements.

For example, in my healthcare RCM portfolio project, one business requirement could be:

‘The system should verify a patient's insurance eligibility before claim processing.’

In the FRD, I would define the functionality in more detail. For example, the system should allow the user to enter or retrieve patient and insurance information, send the required information to the eligibility service, receive the response, display the eligibility status, and handle scenarios such as inactive coverage, missing information, API timeout, or an error response.

I would also document business rules, validations, data requirements, workflows, integrations, and expected system behavior where applicable.

So, in simple terms:

BRD tells us what the business needs and why.
FRD tells us how the system should function to meet those needs.”

Think of FRD as:

"Exactly what should the system do?"

Typical FRD sections:

1. Purpose
2. Scope / Out of Scope
3. Actors / Users / User Roles
4. Functional Requirements ⭐
5. Non-Functional Requirements
6. Business Rules
7. Process Flows / Workflows
8. Use Cases
9. Screen / UI Behavior
10. Input / Output Requirements
11. Validations
12. Error Handling
13. Integration Requirements
14. Data Requirements
15. Reports / Notifications
16. Permissions / Access Control
17. Assumptions
18. Dependencies

Functional Requirements are the heart of the FRD.

For example, in your Healthcare RCM project:

Requirement: Insurance Eligibility Verification

The FRD could define:

FR-01: The system shall allow an authorized user to initiate insurance eligibility verification for a patient.

Then you describe:

- Input: Patient ID, payer, member ID
- Process: Send eligibility request to payer API
- Response: Active / Inactive / Unknown
- Validation: Member ID cannot be blank
- Error handling: API timeout → retry → manual follow-up
- Permission: Only authorized billing users can initiate verification
- Integration: Insurance eligibility API
- Business rule: Eligibility must be verified before claim submission

That's what makes an FRD detailed and functional, rather than just a list of business needs.

⚠️ Small distinction from BRD

Some sections can appear in both BRD and FRD, such as:

- Scope
- Business rules
- Assumptions
- Dependencies
- Stakeholders/users

The level of detail is different.

BRD:

“The business needs automated insurance eligibility verification.”

FRD:

“The system shall allow the billing user to initiate eligibility verification, send the required data to the payer API, process the response, display the eligibility status, and handle timeout/error scenarios.”

Interview answer if asked "What does your FRD contain?"

You can say:

“Typically, my FRD would include scope, actors and user roles, functional and non-functional requirements, business rules, workflows, use cases, UI behavior, validations, error handling, integration and data requirements, reports, permissions, assumptions, and dependencies. The exact template depends on the organization's documentation standards.”

Example 2

Requirement:

User should be able to submit a claim.

FRD could specify:

Input:

- Patient ID
- Insurance ID
- Service date
- Diagnosis
- Procedure
- Charges

Validation:

- Patient ID mandatory
- Insurance must be active
- Service date cannot be future date

System behavior:

- Generate claim ID
- Validate required fields
- Submit claim
- Display confirmation
- Update status to "Submitted"

Error:

If insurance is inactive, claim submission must be blocked and an appropriate error message displayed.

That's functional requirement detail.

13a What is SRS? — Interview-ready explanation

SRS stands for Software Requirements Specification.

It is a document that describes the complete set of requirements for a software system—what the system should do, how it should behave, and the constraints it must follow.

A simple way to understand it:

BRD = What business needs and why
FRD = What functions the system should perform
SRS = Complete software requirements specification

Interview-ready answer

“SRS stands for Software Requirements Specification.

It is a detailed document that specifies the requirements of a software system. It acts as a common reference for the business, BA, developers, QA, and other stakeholders.

An SRS can include functional requirements, non-functional requirements, business rules, user roles, workflows, data requirements, integrations, validations, error handling, security requirements, performance requirements, and other system constraints.

For example, in my healthcare RCM portfolio project, if we need an insurance eligibility feature, the SRS could specify what information the user provides, how the system communicates with the eligibility service, what response the system should display, what happens when the patient is inactive, and how API errors or timeouts should be handled. It could also specify non-functional requirements such as security, performance, and availability.

So, in simple terms, SRS provides a detailed specification of what the software must do and the conditions under which it must operate.”

BRD vs FRD vs SRS

Document	Main focus	Simple question
BRD	Business requirements	Why do we need it? What does the business need?
FRD	Functional requirements	What should the system/function do?
SRS	Complete software requirements	What exactly must the software do and under what conditions?

⚠️ Important for interviews: Organizations don't all use these documents in exactly the same way. Some companies combine FRD/SRS, while Agile teams may capture much of the detail in epics, user stories, acceptance criteria, NFRs, and supporting documentation.

So don't say *"BRD always comes first, then FRD, then SRS in every company."* Say "The documentation structure depends on the organization's methodology and standards."

14. What is a Use Case?

Interview-ready answer

"A Use Case describes how a user or another system interacts with a system to achieve a specific business goal.

It focuses on the interaction between the actor and the system and describes the steps involved, including the normal flow and possible alternative or exception flows.

For example, in my healthcare RCM portfolio project, one use case could be 'Verify Insurance Eligibility.'

The actor could be a Billing User. The billing user enters or retrieves the patient's insurance information and initiates eligibility verification. The system validates the information, sends the request to the eligibility service, receives the response, and displays the eligibility status.

The use case would also define alternative scenarios. For example, if the insurance information is missing, the system should show a validation message. If the eligibility API times out, the system should follow the defined retry or fallback process.

A typical use case includes the use case name, actor, goal, preconditions, trigger, main flow, alternative or exception flows, and postconditions.

So, in simple terms, a use case describes how an actor interacts with the system to accomplish a specific goal."

🧠 *Easy way to remember*

Use Case = Actor + System + Goal + Interaction

Example:

Actor: Billing User

↓

Initiates eligibility verification

↓

System validates information

↓

System calls eligibility service

↓

System receives response

↓

Eligibility status displayed

If interviewer asks: "What is the difference between a Use Case and a User Story?"

A strong short answer is:

"A User Story describes a user need and the value they want, while a Use Case describes the interaction and detailed steps between the actor and the system to achieve that goal."

For example:

User Story:

"As a billing user, I want to verify insurance eligibility so that I can confirm coverage before claim processing."

Use Case:

Describes exactly how the billing user initiates verification, how the system processes it, the response, alternative flows, and exception scenarios**.**

A use case describes example2:

How an actor interacts with the system to achieve a goal.

Three RCM examples

1. Verify Insurance Eligibility
    - Actor: Billing staff
    - Goal: Verify patient's insurance
2. Submit Claim
    - Actor: Billing staff/system
    - Goal: Submit claim to payer
3. Manage Denial
    - Actor: AR specialist
    - Goal: Review and resolve denied claim

---

15. Functional vs Non-Functional Requirements

Functional vs Non-Functional Requirements — Interview-ready answer

Interview-ready answer

“Functional and non-functional requirements describe two different aspects of a system.

Functional requirements define what the system should do. They describe the features, functions, business rules, workflows, inputs, outputs, validations, and system behavior.

For example, in a healthcare RCM system, a functional requirement could be:
‘The system shall allow a billing user to verify a patient's insurance eligibility through the eligibility service and display the eligibility status.’

Non-functional requirements define how well the system should perform or the conditions under which it should operate. They typically cover areas such as performance, security, availability, scalability, usability, and reliability.

For example:
'The eligibility response should be displayed within 3 seconds under normal operating conditions.'

Another example could be:
'Only authorized users should be able to access patient insurance information.'

So, in simple terms:

Functional requirement = What the system does.

Non-functional requirement = How well the system does it or what quality/constraint it must meet.

As a BA, I make sure both types are clearly defined because a system can have all the required features but still fail if it is too slow, insecure, unreliable, or difficult to use."

🧠 *Easy way to remember*

Functional → WHAT

Login, search, calculate, submit claim, generate report, send notification.

Non-functional → HOW WELL

Fast, secure, available, reliable, scalable, usable.

Healthcare RCM example

Type	Example
Functional	System shall submit a claim to the payer.
Functional	System shall display claim status.
Functional	System shall validate required member information.
Non-functional	Claim status should load within 3 seconds.

Type	Example
Non-functional	Only authorized billing users can access PHI.
Non-functional	System should be available 99.9% of the time.

Interview tip: Avoid saying NFRs are simply “technical requirements.” Security, performance, availability, usability, and reliability are common NFRs, but they describe quality attributes or constraints on the system, not necessarily implementation technology.

---

16. What are User Stories?

Interview-ready answer

“A User Story is a short description of a requirement written from the perspective of the user. It explains who needs something, what they need, and why they need it.

In Agile, we commonly write a user story in this format:

As a [user], I want [functionality], so that [business value].

For example, in my healthcare RCM portfolio project:

‘As a billing user, I want to verify a patient's insurance eligibility, so that I can confirm coverage before submitting a claim.’

The user story focuses on the user's need and business value. We then add acceptance criteria to define the conditions that must be satisfied for the story to be considered complete.

As a BA, I help understand the requirement, discuss it with stakeholders and the team, break larger requirements into manageable stories, add business rules and acceptance criteria, and clarify the story during development and testing.

A good user story should be clear, valuable, understandable, testable, and small enough to be delivered within an appropriate sprint or delivery cycle.

So, in simple terms:

User Story = Who needs it + What they need + Why they need it.”

🧠 *Easy way to remember*

As a [WHO] → I want [WHAT] → so that [WHY]

Example:

As a billing user,
I want to verify insurance eligibility,
so that I can confirm coverage before claim submission.

User Story 1 — Insurance Verification

As a billing executive,
I want to verify a patient's insurance eligibility,
so that I can confirm coverage before submitting a claim.

User Story 2 — Claim Submission

As a billing executive,
I want to submit a validated claim electronically,
so that the claim can be processed by the payer.

User Story 3 — Denial Management

As an AR specialist,
I want to view and categorize denied claims,
so that I can prioritize follow-up actions and improve collections.

If interviewer asks: “Is a User Story a requirement?”

Say:

“Yes, a user story represents a requirement from the user's perspective, but it is intentionally lightweight. The details are usually refined through acceptance criteria, business rules, discussions, and other supporting documentation.”

And remember the difference:

User Story → What the user needs and why

Acceptance Criteria → Conditions that must be met for the story to be accepted

Use Case → Detailed interaction between the actor and system

---

17. How do you define Acceptance Criteria?

Interview-ready answer

“Acceptance Criteria are the **specific conditions that a user story must satisfy to be accepted as complete**.

As a BA, I define acceptance criteria based on the **expected business behavior, business rules, positive scenarios, negative scenarios, and exception scenarios**. I make sure the criteria are **clear, specific, measurable, and testable**, so that developers understand what needs to be built and QA can use them to validate the expected behavior.

Where appropriate, I use the **Given-When-Then format**.

For example, if the user story is:

‘As a billing user, I want to verify a patient's insurance eligibility so that I can confirm coverage before submitting a claim.’

The acceptance criteria could be:

1. Positive scenario

Given the patient has a valid insurance ID,
When the billing user requests eligibility verification,
Then the system should display the patient's eligibility status.

2. Negative scenario

Given the insurance ID is invalid,
When the billing user submits the verification request,
Then the system should display an appropriate validation message.

3. Exception scenario

Given the eligibility service is unavailable,
When the billing user requests verification,
Then the system should follow the defined retry or fallback process and display an appropriate status.

So, in simple terms:

User Story tells us what the user needs and why.
Acceptance Criteria tells us what conditions must be met for that story to be accepted.

I also review the acceptance criteria with the Product Owner, business stakeholders, developers, and QA as needed to make sure everyone has a common understanding of the expected behavior.

So, in simple terms, I define acceptance criteria by asking:

'What conditions must be satisfied before we can say that this user story has been successfully completed?'"

I review the acceptance criteria with the Product Owner, relevant business stakeholders, developers, and QA as needed to make sure everyone has the same understanding.

So, in simple terms:

User Story tells us what the user needs and why.
Acceptance Criteria tells us what conditions must be met for that story to be accepted."

Acceptance criteria define:

When can we say that this user story is successfully completed?

Strong interview answer

"I define acceptance criteria based on the expected business behavior, business rules, positive scenarios, negative scenarios and exception scenarios. I generally use Given-When-Then format where appropriate."

17a What is Definition of Done (DoD)? — Interview-ready answer

Interview-ready answer

"Definition of Done, or DoD, is a shared checklist or set of criteria that defines when a user story or increment can be considered completely finished.

As a BA, I make sure the DoD is understood by the team and that the story meets both the acceptance criteria and the team's agreed Definition of Done before it is considered complete.

For example, in a healthcare RCM project, the Definition of Done for a user story might include:

- Development is completed.
- Code has been reviewed according to the team's process.
- Unit and functional testing is completed.
- Acceptance criteria are successfully met.
- QA testing is completed with no critical or blocking defects.
- Required documentation is updated.
- The Product Owner or appropriate stakeholder has accepted the story, based on the team's process.

The exact DoD can vary from one organization or team to another.

The key difference is that Acceptance Criteria are specific to a particular user story, whereas the Definition of Done is a common completion standard applied to stories or increments.

So, in simple terms:

Acceptance Criteria = What must be true for this specific story to be accepted.

Definition of Done = What must be completed for the story to be considered fully done."

🧠 Easy way to remember

Acceptance Criteria → Story-specific

"Did this particular feature behave as expected?"

Definition of Done → Team-wide completion standard

"Have we completed everything required to call this story Done?"

⭐ Common follow-up

Interviewer: "Can a story meet its acceptance criteria but still not be Done?"

Answer:

"Yes. For example, the functionality may meet all the acceptance criteria, but if QA testing, required code review, or another agreed DoD item is still incomplete, the story should not be marked as Done."

18. Root Cause Analysis

Root Cause Analysis (RCA) — Interview-ready answer

Interview-ready answer

“Root Cause Analysis, or RCA, is a structured approach used to identify the underlying reason why a problem occurred, rather than just fixing the immediate symptom.

As a BA, I use RCA when there is a recurring issue, process failure, production incident, defect, or business problem. I first understand and clearly define the problem, collect the relevant data, identify possible causes, analyze the process and dependencies, and then determine the root cause. After that, I work with the relevant stakeholders and technical team to identify corrective and preventive actions.

For example, in a healthcare RCM project, suppose we notice that a large number of claims are being rejected because of missing or incorrect insurance information.

Instead of simply correcting each rejected claim, I would analyze the process and ask why the information is missing or incorrect.

For example:

Why are claims being rejected?
→ Missing insurance information.

Why is insurance information missing?
→ It was not captured correctly during registration.

Why wasn't it captured correctly?
→ The registration process does not validate the required insurance fields.

Why is there no validation?
→ The business requirement did not define mandatory insurance-field validation.

The root cause may therefore be a gap in the requirement and registration validation process, rather than simply an individual data-entry error.

I would then recommend an appropriate corrective action, such as defining the required business rules and adding validation to the registration process, and then verify whether the rejection rate improves.

Common RCA techniques include 5 Whys, Fishbone/Ishikawa Diagram, Pareto Analysis, and process analysis.

So, in simple terms:

RCA means finding the real underlying cause of a problem so that we can prevent the problem from happening again, rather than repeatedly fixing the symptom.”

🧠 *Easy interview formula*

Problem → Collect facts → Analyze causes → Find root cause → Corrective action → Prevent recurrence

RCA means:

Finding the underlying reason why a problem occurred instead of fixing only the symptom.

Example2

Problem:

Claims are being rejected.

Don't immediately say:

"Developer should fix the claim API."

Ask:

Why?

Claim rejected
↓
Missing insurance information
↓
Insurance data not validated
↓
Eligibility API response not mapped correctly
↓
Integration mapping issue

Techniques

- 5 Whys
- Fishbone/Ishikawa
- Pareto analysis

- Process analysis

📌 SQL

19. "I see SQL in your CV. What do you do using SQL?"

For a BA, don't pretend you're a DBA.

A good answer:

As a BA, I use SQL mainly for data validation, requirement analysis and troubleshooting rather than database administration.

For example, I can query tables to verify whether the data displayed in an application matches the database.

I use SELECT queries, WHERE conditions, JOINs, GROUP BY, aggregate functions and subqueries.

I also use SQL to validate business rules, investigate data discrepancies and support QA or UAT when a business issue requires checking the underlying data.

---

20. Primary Key vs Foreign Key

Primary Key vs Foreign Key — Interview-ready answer

Interview-ready answer

“A Primary Key is a column, or combination of columns, that uniquely identifies each record in a table. It must contain unique values and cannot contain NULL values.

A Foreign Key is a column that references the Primary Key of another table. It is used to establish a relationship between two tables and maintain referential integrity.

For example, in a healthcare RCM system, suppose we have two tables:

Patient

Patient_ID	Patient_Name
101	John
102	Sarah

Here, Patient_ID is the Primary Key because it uniquely identifies each patient.

Now suppose we have a Claims table:

Claim_ID	Patient_ID	Claim_Amount
C001	101	500
C002	102	750

Here, Claim_ID is the Primary Key of the Claims table, while Patient_ID is a Foreign Key because it references Patient_ID in the Patient table.

This relationship allows us to connect a claim to the correct patient.

So, in simple terms:

Primary Key = Uniquely identifies a record in its own table.

Foreign Key = Connects one table to another by referencing a key in another table."

🧠 Easy way to remember

Primary Key → Identity

"Who is this record?"

Foreign Key → Relationship

"Which record in another table does this belong to?"

⭐ Common follow-up

Interviewer: "Can a Foreign Key contain NULL?"

Answer:

"Yes, a foreign key can contain NULL if the relationship is optional and the database design allows it. But if the relationship is mandatory, the foreign key can be defined as NOT NULL."

Primary Key

Uniquely identifies a record.

Example:

patient_id

Foreign Key

Connects one table to another.

Example:

claim.patient_id

connects to

patient.patient_id

---

21. SQL Joins

Suppose:

Patient

patient_id	name
1	Raj
2	Amit

Claim

claim_id	patient_id
101	1
102	1

INNER JOIN

Only matching records.

LEFT JOIN

All records from left table + matching right records.

RIGHT JOIN

All records from right table + matching left records.

FULL OUTER JOIN

All records from both tables.

For interviews, INNER JOIN and LEFT JOIN are especially important for a BA.

---

22. DELETE vs TRUNCATE vs DROP

DELETE	TRUNCATE	DROP
Removes selected rows	Removes all rows	Removes table
WHERE allowed	WHERE not allowed	Table structure removed
Table remains	Table remains	Table doesn't remain
DML	Usually DDL	DDL

Simple memory:

DELETE = rows

TRUNCATE = all rows

DROP = table itself

---

23. SQL — Maximum salary

```
SELECT MAX(salary)
FROM employee;
```

Second highest salary

```
SELECT MAX(salary)
FROM employee
WHERE salary < (
   SELECT MAX(salary)
   FROM employee
);
```

A more interview-friendly approach:

SELECT DISTINCT salary

FROM employee

ORDER BY salary DESC

LIMIT 2;

For SQL Server, you'd use TOP or DENSE_RANK() instead, depending on the requirement.

📌 API

24. What is an API?

Interview-ready answer

“API stands for Application Programming Interface. It is a mechanism that allows two different software systems to communicate and exchange data with each other.

For example, in a healthcare RCM system, our application may need to verify a patient's insurance eligibility. Instead of manually checking the payer's system, the RCM application can communicate with the payer's eligibility service through an API.

The RCM system sends an API request containing the required information, such as patient and insurance details. The payer system processes the request and sends back an API response, such as active, inactive, or other eligibility information.

As a BA, when working with an API requirement, I focus on understanding the business purpose, endpoint, request and response payload, authentication, required fields, data mapping, business rules, validations, error handling, and dependencies. I also work with developers and QA to clarify the expected behavior and support API testing.

For example, I may use tools such as Postman to understand and validate API requests and responses from a testing perspective.

So, in simple terms:

API is a bridge that allows one system to communicate with another system and exchange information in a defined way.”

API = Application Programming Interface

Simple explanation:

API allows two software systems to communicate with each other.

Example:

RCM System

↓

API request

↓

Insurance/Payer System

↓

API response

↓

RCM System

Advantages

- **System integration**
- **Automation**
- **Reusability**
- **Data exchange**
- **Reduced manual work**
- **Real-time communication**

Common API styles/types you'll hear

- **REST**
- **SOAP**
- **GraphQL**
- **Webhooks**

---

25. REST vs SOAP

REST	SOAP
Architectural style	Protocol
Usually JSON	XML
Lightweight	More formal/heavier
Easy for web/mobile APIs	Common in enterprise/legacy systems
HTTP methods	SOAP operations
Flexible	Strict standards

Interview answer

"REST is an architectural style generally using HTTP and commonly JSON, whereas SOAP is a protocol based primarily on XML with defined standards and contracts."

📌 AGILE & SCRUM

26. What is Agile?

Agile is a flexible and iterative way of working where we develop and deliver a product in small, manageable increments, rather than building the entire product at once.

The main idea of Agile is to deliver value early, get continuous feedback from customers or stakeholders, collaborate closely with the team, and adapt to changing requirements.

For example, in a Healthcare RCM project, instead of developing the complete system first, we can deliver it step by step:

- First, patient registration
- Then, insurance eligibility
- Then, claim submission
- Then, denial management

After each increment, we get feedback and use that feedback to improve or reprioritize the next work.

As a Business Analyst, I work closely with stakeholders and the product team to understand requirements, analyze and prioritize them, create or support user stories and acceptance criteria, clarify requirements with developers and QA, and support testing and UAT.

In simple terms, Agile means:
Build → Deliver → Get Feedback → Adapt → Improve → Repeat.

Likely follow-up: "What is the difference between Agile and Scrum?"

Agile is the broader mindset and set of principles, while Scrum is a specific framework used to apply Agile principles.

Agile is based on the Agile Manifesto, which has 4 core values:

1. Individuals and interactions over processes and tools
   This means effective communication and collaboration are more important than simply following tools or processes.
2. Working software over comprehensive documentation
   Documentation is still important, but the priority is to deliver something that actually works and provides value.
3. Customer collaboration over contract negotiation
   Agile encourages continuous collaboration with the customer or business stakeholders rather than relying only on an initial agreement.
4. Responding to change over following a plan
   Requirements can change as we learn more, so Agile teams adapt instead of rigidly following the original plan.

Agile also follows principles such as early and continuous delivery, welcoming changing requirements, frequent delivery, continuous improvement, close collaboration, and maintaining a sustainable pace.

In practice, Agile usually involves breaking a large product or requirement into smaller, manageable pieces, prioritizing them, developing them incrementally, getting feedback, and continuously improving.

Don't say only:

Agile means flexibility.

Better:

Agile is an iterative and incremental approach to delivering software where teams continuously deliver value, receive feedback and adapt to changing requirements.

Advantages

- Faster feedback
- Early value delivery
- Adaptability
- Continuous improvement
- Better stakeholder collaboration
- Reduced risk

---

27. What is Scrum?

Scrum is a framework for developing and delivering products iteratively and incrementally.

Scrum has:

It has three main pillars:

- Transparency – everyone has a clear understanding of the work and its status.
- Inspection – we regularly review the product and progress.
- Adaptation – based on what we learn, we adjust the plan or approach.

Roles/accountabilities

- Product Owner

- Scrum Master
- Developers

Events

- Sprint
- Sprint Planning
- Daily Scrum
- Sprint Review
- Sprint Retrospective

Artifacts

- Product Backlog
- Sprint Backlog
- Increment

As a Business Analyst, depending on the organization and team structure, I support Scrum by understanding and analyzing business requirements, helping refine backlog items, creating or supporting user stories and acceptance criteria, clarifying requirements with Developers and QA, participating in Sprint Planning and Backlog Refinement, and supporting UAT and stakeholder feedback.

For example, in a Healthcare RCM project, the Product Backlog may contain features such as insurance eligibility, claim submission, denial management, and payment posting. The team prioritizes the work, selects appropriate items for a Sprint, develops a usable increment, gets stakeholder feedback during the Sprint Review, and uses that feedback to improve the next Sprint.

In simple terms:

Scrum = Agile framework + Roles/Accountabilities + Events + Artifacts + Values + Incremental delivery + Continuous inspection and adaptation.

---

28. What is Product Backlog and Sprint Backlog?

The Product Backlog is an ordered list of everything that may be needed to improve or build the product. It can contain features, user stories, bug fixes, technical work, improvements, and other product-related work.

The Product Owner is accountable for managing the Product Backlog, including its ordering, while the Scrum Team contributes to understanding and refining the work.

The Sprint Backlog is the set of Product Backlog Items selected for the current Sprint, along with the plan for delivering them and achieving the Sprint Goal. It is managed by the Developers and can be updated during the Sprint as they learn more.

For example, in a Healthcare RCM project, the Product Backlog might contain:

- Insurance eligibility
- Claim submission
- Claim status tracking
- Denial management
- Payment posting
- AR management

During Sprint Planning, the team may select insurance eligibility and related stories for the current Sprint.

So:

Product Backlog
→ Contains the broader, ordered list of product work
→ Continuously evolves and is refined

Sprint Backlog
→ Contains the selected work for the current Sprint + the plan to achieve the Sprint Goal
→ Can be updated during the Sprint

As a BA, I support this process by helping clarify requirements, refining backlog items, defining acceptance criteria, identifying business rules and dependencies, and answering questions from Developers and QA.

In simple terms:

Product Backlog = What may need to be built for the product.
Sprint Backlog = What the team is working on now and the plan to achieve the Sprint Goal.

Product Backlog

Contains the ordered list of product work.

Sprint Backlog

Contains the work selected for the current Sprint, plus the team's plan for delivering it.

---

29. Scrum Pillars

Scrum is based on empiricism, which means we make decisions based on observation, experience, and what we learn rather than assuming everything can be predicted upfront.

Scrum has three pillars:

1. Transparency

Transparency means that important information about the product, process, and work is visible and understood by everyone involved.

For example, the Product Backlog, Sprint Backlog, Sprint Goal, and Definition of Done should be clear to the Scrum Team.

BA example: As a BA, I make sure requirements, acceptance criteria, business rules, and dependencies are clearly documented and understood by the team.

2. Inspection

Inspection means we regularly examine the product, progress, and way of working to identify problems or opportunities for improvement.

For example, during the Daily Scrum, the team inspects progress toward the Sprint Goal, and during the Sprint Review, stakeholders inspect the product Increment.

3. Adaptation

Adaptation means that when inspection identifies a problem or new information, the team adjusts its approach or plan.

For example, if stakeholders provide new feedback during the Sprint Review, the Product Owner may reprioritize the Product Backlog for future Sprints.

As a BA, I support adaptation by analyzing feedback, clarifying changed requirements, identifying impacts and dependencies, and helping convert valid changes into clear backlog items.

Healthcare RCM example

Suppose we are developing an insurance eligibility feature:

Transparency → Everyone understands the requirement, acceptance criteria, and Sprint Goal.

Inspection → During testing and Sprint Review, we discover that some insurance responses are not being handled correctly.

Adaptation → The team updates the requirement/business rules and adds the required scenarios to the backlog.

In simple terms:

Transparency → Make it visible
Inspection → Check what is happening
Adaptation → Change/improve based on what we learn

These three pillars allow Scrum teams to learn continuously and respond effectively to change.

⭐ Easy interview memory

TIP: T-I-A

T → Transparency → See it
I → Inspection → Check it
A → Adaptation → Improve it

Likely follow-up:
Q: Why are these called pillars?

Because they support empirical process control in Scrum. Without transparency, we cannot inspect effectively; without inspection, we don't know what needs to change; and without adaptation, inspection doesn't lead to improvement.

📌 BEHAVIORAL QUESTIONS

30. Rate yourself 1–10 in SQL/API/Power BI

Don't say:

SQL = 10/10.

That creates a trap.

For your current level, I'd recommend something like:

I would rate myself around 7 out of 10 in SQL from a Business Analyst perspective. I am comfortable with querying, joins, filtering, aggregation and data validation.

For APIs, I would rate myself around 7. I understand REST APIs, request-response flow, authentication concepts, JSON payloads, status codes and integration scenarios, and I can analyze API requirements and troubleshoot basic integration issues.

For Power BI, I would rate myself around 6. I can work with datasets, create relationships, build visuals, use filters and create business dashboards. I am continuing to strengthen advanced DAX and data modelling.

Rule: Never claim 9/10 unless you can handle deep technical follow-up questions.

---

31. How do you handle changing requirements?

Use:

Understand → Impact → Prioritize → Approve → Update → Communicate

Answer

"I don't reject changing requirements automatically. I first understand why the requirement changed and what business value it provides. Then I perform an impact analysis covering scope, timeline, cost, dependencies, development and testing. I discuss the impact with the Product Owner/business stakeholders and prioritize the change. Once approved, I update the relevant requirements, backlog items and acceptance criteria and communicate the change to the affected teams."

Excellent BA answer.

---

32. Project is delayed but business wants faster delivery

This is a very important scenario question.

Don't say:

"I'll ask developers to work faster."

Instead:

Assess → Identify bottleneck → Prioritize → Re-plan → Communicate

Example:

"First, I would understand why the project is delayed—requirements, development, dependencies, testing, resources or scope. Then I would identify the critical path and discuss options with the Product Owner and technical team. We could prioritize the MVP, defer lower-priority features, remove unnecessary scope, parallelize activities where possible, or add resources if appropriate. I would communicate the realistic impact rather than promising an unrealistic delivery date."

---

33. How do you ensure solution meets requirements after implementation?

Use:

Requirement → Acceptance Criteria → Testing → UAT → Production Validation → Feedback

Answer:

"I maintain traceability from requirements to acceptance criteria and test scenarios. Before release, I ensure UAT validates the business scenarios. After deployment, I support production validation and collect user feedback. If there are gaps, I analyze them against the original requirement and acceptance criteria and create defects or enhancement items accordingly."

---

34. Most challenging project?

Don't invent some dramatic production incident.

For your portfolio, you can discuss integration complexity or changing requirements.

Example:

"One of the more challenging areas in my healthcare RCM project was designing the claim-status integration because the business expected near-real-time status updates while the external system might not always provide an immediate response."

Then explain:

Problem → Analysis → Webhook → Polling fallback → Error handling → Business outcome

This connects nicely with your API preparation.

📌 BUSINESS PROCESS & TOOLS

35. Wireframe vs Prototype

A wireframe focuses mainly on structure and layout, while a prototype demonstrates interaction and user flow. A wireframe is like a blueprint, whereas a prototype is closer to a working model of the user experience.

A wireframe is a basic visual representation of a screen or page. It focuses mainly on the layout, structure, and placement of elements, rather than colors, detailed design, or full functionality.

A prototype is a more interactive representation of the product that allows users and stakeholders to experience how the system will work and navigate between screens.

For example, in a Healthcare RCM application, suppose we are designing an Insurance Eligibility screen.

A wireframe might show:

- Patient information section
- Insurance information section
- “Verify Eligibility” button
- Eligibility status area
- Error message area

It mainly answers: “What should be on the screen and where?”

A prototype could allow the stakeholder to click “Verify Eligibility”, move to the results screen, see an Active/Inactive status, and experience the navigation and interaction.

It answers: “How will the user interact with the system?”

As a BA, I may use wireframes during requirement analysis to clarify screen layout and business requirements. Prototypes are useful when we need to validate the user experience and workflow with stakeholders before development.

In simple terms:

Wireframe = Structure / Layout
Prototype = Interaction / Experience

A wireframe can be static and low-fidelity, while a prototype can range from simple to highly realistic and interactive.

The exact level of detail depends on the project and the tool being used.

⭐ Easy interview memory

	Wireframe	Prototype
Main purpose	Screen structure	User interaction
Detail	Low/basic	Can be low to high
Interaction	Usually static	Usually interactive
Focus	What goes where?	How does it work?
BA use	Clarify requirements	Validate workflow/UX

Easy formula:

Wireframe = Blueprint 📐
Prototype = Model you can interact with 🖱️

Likely follow-up: "Is a prototype the final product?"

No. A prototype is a representation used to validate requirements, workflow, and user experience before or during development. It is not necessarily the final production UI.

Memory trick

Wireframe = How the screen is structured

Prototype = How the user interacts

---

36. What do you do in Power BI?

For a BA:

"I use Power BI to convert business data into dashboards and reports that help stakeholders monitor KPIs and make decisions."

What do you do in Power BI?

As a Business Analyst, I use Power BI to analyze business data and convert it into meaningful dashboards and insights that help stakeholders make decisions.

My involvement typically starts with understanding the business reporting requirements—what KPIs stakeholders need, what data they want to see, who will use the dashboard, and what decisions they need to make.

Then I work with the data by:

- Understanding the data sources and required fields
- Performing basic data validation and cleaning
- Defining KPIs and business calculations
- Creating or supporting the data model and relationships
- Building visualizations such as charts, tables, cards, and trend analysis
- Adding filters, slicers, and drill-downs for better analysis
- Validating the dashboard numbers against the source data
- Gathering stakeholder feedback and refining the dashboard

For example, in a Healthcare RCM dashboard, I could create KPIs such as:

- Total claims submitted
- Clean claim rate
- Denial rate
- Total collections
- Accounts Receivable
- Days in AR

- Payer-wise performance
- Denial reasons and trends

The dashboard could allow stakeholders to filter the data by payer, facility, date, claim status, or denial reason and identify areas that need attention.

As a BA, my main focus is not just creating visualizations. I focus on understanding what the business needs to measure, defining the correct KPIs and business rules, validating the data, and ensuring the dashboard answers the business questions.

In simple terms:

Business requirement → Data → KPI → Analysis → Dashboard → Business insight → Decision

Likely follow-up: *"What is the difference between Power BI and Excel?"*

Excel is excellent for spreadsheet-based analysis and smaller/manual datasets, while Power BI is designed more for interactive dashboards, data modeling, visualization, and reporting across larger or multiple data sources.

Typical workflow:

Business requirement

↓

Identify KPIs

↓

Collect data

↓

Clean/transform data

↓

Create data model

↓

Create relationships

↓

Create measures/DAX

↓

Create visuals

↓

Add filters/slicers

↓

Validate numbers

↓

Publish/share dashboard

Healthcare RCM example

Dashboard could show:

- Total claims
- Paid claims
- Denied claims
- Claim acceptance rate
- Denial rate
- Outstanding AR
- Days in AR
- Payer-wise performance
- Denial reason trends

⭐ 37. What is Gap Analysis?

This is one of the most important BA concepts.

Gap Analysis means:

Comparing the current state (AS-IS) with the desired future state (TO-BE) to identify what is missing or needs to change.

Example

Hospital currently:

AS-IS

Billing staff manually checks insurance eligibility.

Desired:

TO-BE

System automatically verifies insurance through an API.

Gap

Manual eligibility verification

needs to become

Automated eligibility verification

BA identifies:

- Process gap
- Technology gap
- Data gap
- Skill gap
- Compliance gap
- Integration gap

Interview-ready answer

Gap Analysis is a technique used to compare the current state with the desired future state and identify the gaps that need to be addressed.

For example, in a healthcare RCM process, suppose the current process requires billing staff to manually verify insurance eligibility. The desired process is to automatically verify eligibility through an API.

The gap is the lack of automated eligibility integration.

As a BA, I would analyze the gap, understand the business impact, identify the required system changes, dependencies and risks, and then convert the identified needs into requirements and actionable backlog items.

🧠 **Your BA Interview Mental Model**

You should be able to connect almost every question to this flow:

BUSINESS PROBLEM

↓

STAKEHOLDERS

↓

REQUIREMENT ELICITATION

↓

AS-IS PROCESS

↓

GAP ANALYSIS

↓

TO-BE PROCESS

↓

BRD / FRD

↓

USE CASE / USER STORY

↓

ACCEPTANCE CRITERIA

↓

BACKLOG

↓

SPRINT

↓

DEVELOPMENT

↓

QA / TESTING

↓

UAT

↓

DEPLOYMENT

↓

PRODUCTION VALIDATION

↓

FEEDBACK / ENHANCEMENT

This is much more important than memorizing 50 isolated definitions.

BUSINESS ANALYST INTERVIEW MASTER ANSWER BANK

Your Interview Strategy

For almost every answer, think:

Business Problem → Requirement → Analysis → Solution → Development → Testing → UAT → Deployment → Feedback

And for project questions:

Problem → Users → AS-IS → Gap → TO-BE → Requirements → Solution → BA contribution → Result

---

📌 SECTION 1 — BASIC BA QUESTIONS

1. Tell me about yourself / Walk me through your profile

Interview-ready answer

I have a background in pharmacy, with a Master's degree in Pharmacology, and I started my professional career in the pharmaceutical industry.

I have experience in areas such as regulatory affairs and business development, where I worked with business stakeholders, documentation, coordination and process-related activities.

Over time, I developed an interest in technology and decided to transition into Business Analysis. To build practical IT BA skills, I worked on hands-on healthcare projects covering Hospital Management, Clinical Trial Management, Healthcare Revenue Cycle Management and Healthcare Analytics.

Through these projects, I have practiced requirement elicitation, stakeholder analysis, BRD and FRD documentation, process modelling, BPMN and UML, use cases, user stories, acceptance criteria, Jira, SQL, API concepts, UAT and Agile/Scrum practices.

My pharmaceutical and healthcare background gives me an additional advantage when working on healthcare technology products because I understand the business and domain context.

I am now looking for a Business Analyst opportunity where I can combine my healthcare domain knowledge, business analysis skills and technology understanding to contribute to real-world projects.

Likely follow-up

"You don't have IT BA experience. Why should we hire you?"

Answer:

Although I am transitioning into IT, I have deliberately built practical BA skills through hands-on projects rather than learning only theory. I also bring prior experience working with business stakeholders, documentation and pharmaceutical processes. So I understand both the business side and the BA methodology, and I am now looking for an opportunity to apply these skills in a real project environment.

---

2. What are your roles and responsibilities as a BA?

My primary responsibility as a BA is to understand the business problem and translate it into clear and actionable requirements for the technical team.

My responsibilities include stakeholder identification, requirement elicitation, requirement analysis, BRD and FRD preparation, process modelling, use cases, user stories and acceptance criteria.

In Agile projects, I also participate in backlog refinement, sprint planning and requirement clarification.

During development, I act as a bridge between business stakeholders and the development team.

I support QA by clarifying business scenarios and reviewing requirements against test scenarios.

During UAT, I support business users, clarify expected behavior and help validate whether the solution meets the business requirements.

After deployment, I support business validation and collect feedback for defects or future enhancements.

---

3. What are your day-to-day activities?

I usually start by checking project communication, emails and Jira for any new updates, questions, blockers or requirement changes.

If there is a Daily Scrum, I participate and understand the progress of development and QA and whether there are any requirement-related blockers.

Depending on the project phase, my day may include stakeholder meetings, requirement elicitation, documentation, process modelling, writing or refining user stories, answering developer questions, reviewing test scenarios, supporting UAT or analyzing business issues.

I also make sure that important decisions, requirement changes and stakeholder feedback are properly documented and communicated.

Interviewer may ask:

"What exactly do you do in Jira?"

We'll cover that later.

---

4. How do you gather requirements?

Use this structure:

Prepare → Elicit → Analyze → Validate → Document → Approve

First, I understand the business objective and identify the relevant stakeholders.

Then I select the appropriate elicitation technique such as interviews, workshops, observation, document analysis, questionnaires or brainstorming.

I understand the current AS-IS process, pain points, business rules and exceptions.

Then I identify the desired TO-BE process and analyze the gap between the current and future state.

I document the requirements, validate them with stakeholders and resolve ambiguity or conflicts.

Finally, I obtain stakeholder confirmation or approval and maintain traceability throughout the project.

---

5. How do you prepare for a stakeholder meeting?

Before

- Understand objective
- Identify stakeholders
- Review existing documents
- Understand AS-IS process
- Prepare questions

- Prepare agenda

During

- Understand business problem
- Ask open-ended questions
- Identify requirements
- Capture business rules
- Identify exceptions
- Identify dependencies
- Confirm priorities

After

- MoM
- Action items
- Open questions
- Requirement updates
- Stakeholder validation

Interview answer

I prepare by first understanding the purpose of the meeting and reviewing any available documentation. I identify the right stakeholders and prepare an agenda and questions.

During the meeting, I focus on understanding the business problem, current process, pain points, desired future state, business rules, exceptions, integrations and priorities.

After the meeting, I document the decisions, action items and open questions and update the relevant requirements. I then validate my understanding with the stakeholders.

---

6. What is your role as a BA in testing?

A BA is not normally responsible for executing all technical testing. My responsibility is to ensure that the testing validates the business requirements.

I make sure requirements and acceptance criteria are clear and testable.

I work with QA to clarify business scenarios, business rules, expected outcomes and edge cases.

I may review test scenarios and help determine whether a reported issue is actually a defect or expected behavior.

During UAT, I support business users and validate that the implemented functionality meets the agreed business requirements.

Remember:

BA = Business validation

QA = Quality/testing execution

There can be overlap, but don't say BA replaces QA.

---

7. What is your role in deployment?

I don't normally perform the technical deployment myself. Development or DevOps teams handle that.

My role is to support business readiness and validation.

Before deployment, I ensure UAT is completed and business stakeholders understand the changes.

After deployment, I support smoke or business validation, verify critical functionality and collect production feedback.

If an issue occurs, I help assess its business impact and coordinate with the appropriate technical team.

📌 SECTION 2 — PROJECT QUESTIONS

For your interviews, we'll use **Healthcare RCM** as one of your main project stories because it gives you excellent opportunities to discuss:

- Requirements
- Healthcare
- APIs
- SQL

- User journey
- Agile
- UAT
- Power BI
- Business process
- Integrations
- RCA

---

8. Explain your project

Healthcare RCM Platform

The project was a healthcare Revenue Cycle Management platform designed to help healthcare organizations manage the financial lifecycle of patient services.

The major objective was to improve visibility into claims, payments, denials and accounts receivable and reduce manual effort.

The key users included billing executives, AR specialists, revenue cycle managers and administrators.

The major workflow included patient and insurance information, eligibility verification, charge capture, claim creation, claim submission, payer response, payment posting, denial management and reporting.

As a BA, I focused on understanding the business process, identifying requirements and gaps, designing the TO-BE process, creating functional requirements and Agile user stories, and supporting development, testing and UAT.

---

9. Explain the project workflow and user journey

Patient Registration

↓

Insurance Verification

↓

Service / Charge Capture

↓

Claim Creation

↓

Claim Validation

↓

Claim Submission

↓

Payer Response

↓

Payment Posting

↓

Denial Management

↓

AR Follow-up

↓

Analytics / Reporting

Interview answer

The journey starts when patient and insurance information is captured. The system verifies insurance eligibility.

After the healthcare service is provided, charges and relevant information are captured and a claim is generated.

The claim goes through validation and is submitted to the payer.

The payer sends a response such as accepted, rejected, denied or paid.

Payments are posted when applicable, while denied claims are routed for investigation and follow-up.

Finally, managers use dashboards to monitor claims, denials, collections and outstanding AR.

---

10. What was your contribution as a BA?

Use the **full lifecycle**.

I started by understanding the business problem and identifying stakeholders and user roles.

I analyzed the AS-IS process and identified pain points and gaps.

I worked on the TO-BE process and documented requirements.

I prepared functional requirements, process flows, use cases and user stories with acceptance criteria.

I supported backlog refinement and requirement clarification during development.

I worked with QA to clarify business scenarios and supported UAT.

I also analyzed feedback and identified defects or enhancement requirements.

---

11. What challenges did you face?

Don't give only one. Prepare **three stories**.

Challenge 1 — Changing requirements

A stakeholder requested additional functionality after the initial requirement had already been defined. I first understood the business reason, then performed impact analysis covering scope, development, testing, dependencies and timeline. I discussed the impact with the Product Owner and helped prioritize the change. After approval, I updated the relevant requirements and communicated the change.

Challenge 2 — Requirement ambiguity

Different stakeholders had different interpretations of the same requirement. I organized a clarification discussion, documented the different expectations, identified the business rule that should drive the requirement and obtained agreement before finalizing it.

Challenge 3 — Integration issue

An external system did not always provide an immediate response. I analyzed the expected business behavior and worked with the technical team on an asynchronous integration approach using webhook-based updates with polling as a fallback.

---

12. What solution did you implement on your own?

Be careful with wording.

Don't say:

"I developed the complete system myself."

Instead:

One solution I worked through from a BA and solution-analysis perspective was an asynchronous claim-status update mechanism.

Instead of repeatedly requiring users to manually check claim status, the external system could notify our platform through a webhook when the status changed.

Because external systems may occasionally fail to send the notification, we also considered polling as a fallback mechanism.

This reduced dependence on manual checking and provided a more reliable claim-status update process.

📌 SECTION 4 — REQUIREMENT MANAGEMENT

22. What is requirement elicitation?

Requirement elicitation is the process of discovering and understanding stakeholder needs, expectations, problems and business requirements using techniques such as interviews, workshops, observation, questionnaires and document analysis.

Interview-ready answer

Requirement elicitation is the process of discovering and understanding the needs, expectations, problems and requirements of stakeholders.

As a BA, I use different techniques to understand the current process, identify pain points and determine what the stakeholders actually need from the solution.

The objective is not just to collect what stakeholders say, but to understand the underlying business problem and convert it into clear requirements.

Example

A hospital says:

"We need a better claims system."

I would ask:

- What problems are you facing?
- Where is the current process slow?
- What are users doing manually?
- What information is missing?
- What outcome do you expect?

. What are Requirement Elicitation Techniques?

Common techniques include:

1. **Interviews**
2. **Workshops**
3. **Observation**
4. **Document analysis**
5. **Questionnaires/surveys**
6. **Brainstorming**
7. **Focus groups**
8. **Prototyping**
9. **Interface analysis**

Interview answer

The technique I choose depends on the situation. For individual stakeholder requirements, I may use interviews. For conflicting or complex requirements, workshops are useful. If I need to understand how users actually perform a process, I can use observation. I can also analyze existing documents, reports and systems.

. What is Requirement Analysis?

Requirement analysis is the process of examining, organizing, refining and prioritizing gathered requirements to ensure they are clear, complete, consistent, feasible and aligned with the business objective.

Example

Stakeholder says:

"The dashboard should be fast."

That's vague.

I would clarify:

"What response time would the business consider acceptable?"

It might become:

"The dashboard should load within 3 seconds under normal operating conditions."

That's requirement analysis and refinement.

. What is Requirement Prioritization?

Requirement prioritization means determining which requirements are most important based on factors such as business value, customer impact, risk, urgency, regulatory importance, dependencies and implementation effort.

Common technique

MoSCoW

- Must Have
- Should Have
- Could Have
- Won't Have

Healthcare example

A regulatory requirement may become a **Must Have**, while a cosmetic dashboard enhancement may be a **Could Have**.

. What is a Business Rule?

A business rule is a policy, condition or constraint that defines or controls how a business process or system should behave.

Healthcare example

A claim cannot be submitted if the patient's insurance coverage is inactive.

That's a **business rule**.

Another:

Only authorized billing users can submit claims.

. What is an Assumption?

An assumption is something we consider to be true for planning or analysis purposes, even though it has not necessarily been fully confirmed.

Example:

We assume that the payer will provide claim-status information through an API.

If that assumption turns out to be wrong, it could affect the solution.

. What is a Dependency?

A dependency means one activity, requirement, team or system depends on another item being available or completed.

Example:

Claim submission depends on the insurance eligibility integration being available.

. What is a Constraint?

A constraint is a limitation that restricts how a project or solution can be delivered.

Examples:

- Budget
- Timeline
- Technology
- Resources
- Regulatory requirements
- Existing system limitations

Example:

The project must use the organization's existing authentication platform.

That's a constraint.

. What is a Requirement Change Request?

A requirement change request is a formal request to modify, add or remove an existing requirement after it has already been defined or approved.

As a BA, I would:

Understand → Analyze impact → Prioritize → Get approval → Update documentation → Communicate

---

23. What is requirement validation?

Requirement validation ensures that requirements are correct, complete, consistent, feasible, testable, unambiguous and aligned with the business objective.

Requirement validation is the process of confirming that requirements correctly represent the business need and are suitable for implementation and testing.

I check whether requirements are:

- Correct
- Complete
- Consistent
- Unambiguous

- Feasible
- Testable
- Traceable

Example

If a requirement says:

"System should process claims quickly."

I would validate what **quickly** means and convert it into a measurable requirement.

---

24. What is requirement verification?

A useful interview distinction:

Verification:

Are we documenting the requirement correctly?

Validation:

Are we documenting the right requirement?

---

25. What makes a good requirement?

A good requirement should be:

- Clear
- Complete
- Consistent
- Unambiguous
- Feasible
- Testable
- Traceable
- Prioritized

Strong interview answer

A good requirement should clearly describe the expected business or system behavior without ambiguity. It should be feasible, testable and traceable, and it should have enough detail for development and QA to understand what needs to be delivered.

---

26. What is requirement traceability?

Requirement Traceability means maintaining a relationship between the original business requirement and downstream artifacts such as functional requirements, user stories, test cases and defects.

Requirement traceability is the ability to track a requirement throughout the entire project lifecycle, from its original business need through analysis, development, testing, and final delivery.

The main purpose is to make sure that every requirement is implemented, tested, and delivered, and that we can identify the impact if a requirement changes.

As a BA, I maintain traceability by linking requirements with related business objectives, functional requirements or user stories, acceptance criteria, design/development work, test cases, defects, and UAT results, depending on the project and documentation approach.

For example, in a Healthcare RCM project, suppose we have a requirement:

“The system should verify insurance eligibility before claim submission.”

I would be able to trace this requirement to:

Business Requirement
→ Insurance eligibility verification

User Story
→ Billing user wants to verify eligibility

Acceptance Criteria
→ Valid insurance → eligibility status displayed

Development
→ Eligibility API and UI functionality

Test Case
→ Verify active, inactive, invalid, and API-error scenarios

UAT
→ Business validates the functionality

This helps ensure that the original business requirement has actually been delivered and tested.

Requirement traceability also helps with change impact analysis. For example, if the eligibility requirement changes, I can identify which user stories, APIs, business rules, test cases, and UAT scenarios may be affected.

A common tool used for this is an RTM — Requirements Traceability Matrix.

In simple terms:

Requirement → Design/Development → Test → UAT → Delivery

Traceability answers the question:

"Can I prove where this requirement came from, how it was implemented, how it was tested, and whether it was successfully delivered?"

Example:

BR-001

↓

FR-001

↓

US-001

↓

AC-001

↓

TC-001

↓

UAT-001

This is where **RTM — Requirement Traceability Matrix** becomes important.

⭐ Easy interview memory

Think “Trace the requirement from START → END”

Business Need
↓
Requirement / User Story
↓
Acceptance Criteria
↓
Development
↓
Test Case
↓
Defect/UAT
↓
Delivered

Likely follow-up

Q: What is the difference between Requirement Traceability and RTM?

Requirement traceability is the concept or process of tracking requirements throughout the lifecycle. RTM is the document or matrix commonly used to record those traceability links.

---

27. What is RTM?

RTM is a Requirement Traceability Matrix used to ensure that requirements are mapped to corresponding test cases and ultimately validated.

RTM stands for Requirements Traceability Matrix. It is a document or matrix used to map and track requirements throughout the project lifecycle to make sure that every requirement is properly implemented and tested.

As a BA, I use RTM to establish traceability between the requirement, user story or functional requirement, acceptance criteria, test cases, defects, and UAT, depending on the project's process.

For example, in a Healthcare RCM project, suppose we have a requirement:

“The system should verify insurance eligibility before claim submission.”

In the RTM, I could track it like this:

Requirement ID: BR-001
→ User Story: US-015
→ Acceptance Criteria: AC-015
→ Test Cases: TC-101, TC-102
→ Defect: DEF-25, if applicable
→ UAT Result: Passed

This helps me confirm that the requirement has been covered during development, tested by QA, and validated during UAT.

RTM is also useful for change impact analysis. If a requirement changes, I can identify the related user stories, test cases, defects, and other items that may be affected.

It also helps identify missing requirements, missing test coverage, and unnecessary or unlinked test cases.

The exact RTM format varies by organization. In an Agile project, traceability may be maintained through tools such as Jira or Azure DevOps rather than a separate Excel document.

In simple terms:

RTM = Requirement → Development → Testing → UAT → Delivery

Its main purpose is to make sure nothing required by the business is missed and everything required is properly validated.

Likely follow-up

Q: Who maintains the RTM?

Usually the BA or requirements analyst is responsible for maintaining it, but the exact ownership depends on the organization. QA, developers, Product Owner, and other team members may contribute to keeping the traceability information up to date.

Req ID	**Requirement**	**User Story**	**Test Case**	**UAT**	**Status**
BR-01	Verify insurance	US-01	TC-01	UAT-01	Passed
BR-02	Submit claim	US-02	TC-02	UAT-02	Passed

Interview tip

Don't say:

"RTM is only used by QA."

BA is often heavily involved in maintaining or supporting traceability.

---

28. What is scope?

Scope defines the boundaries of a project or product. It clearly identifies what is included, what is not included, and what the project is expected to deliver.

As a Business Analyst, I help define and clarify scope by understanding business objectives, stakeholder expectations, requirements, processes, and constraints. I also make sure that in-scope and out-of-scope items are clearly documented so everyone has a common understanding.

Scope generally covers areas such as:

- In-scope – what the project will deliver
- Out-of-scope – what the project will not deliver
- Business processes or functionalities covered
- Users or stakeholders affected
- Systems or integrations involved
- Key assumptions, constraints, and dependencies
- Expected deliverables

For example, in a Healthcare RCM project, suppose the objective is to improve claim processing.

In scope:

- Insurance eligibility verification
- Claim creation and validation
- Claim submission
- Claim status tracking
- Denial management

Out of scope:

- Clinical treatment management
- Hospital inventory management
- Pharmacy management

Clearly defining scope helps prevent scope creep, avoids misunderstandings, supports estimation and planning, and keeps the team focused on the agreed business objectives.

If a stakeholder later asks for a new feature, I would not simply add it. I would first analyze the impact on scope, timeline, cost, resources, dependencies, and requirements, and then follow the organization's change-control or backlog-prioritization process.

In simple terms:

Scope = What we will do + What we will not do + What we will deliver.

⭐ Easy interview memory

Think of scope as a boundary line:

Inside the boundary → IN SCOPE ✅
Outside the boundary → OUT OF SCOPE ❌

And remember:

Scope protects the project from uncontrolled expansion.

Likely follow-up: "What is scope creep?"

Scope creep is the uncontrolled addition of new requirements or work to a project without properly evaluating and approving the impact on scope, time, cost, resources, or other constraints.

29. What is scope creep?

Scope creep occurs when additional requirements or functionality are added without proper evaluation and control of their impact on scope, timeline, resources or cost.

What is Scope Creep?

Scope creep is the uncontrolled or unapproved addition of new requirements, features, or work to a project after the scope has already been agreed, without properly evaluating its impact.

As a BA, my responsibility is not to simply reject every new request, but to understand the change, analyze its impact, and make sure it goes through the appropriate change-control or backlog-prioritization process.

For example, in a Healthcare RCM project, suppose the agreed scope includes:

- Insurance eligibility
- Claim submission
- Claim status tracking
- Denial management

During development, a stakeholder asks to add patient appointment scheduling and pharmacy management.

If the team starts building these features without assessing and approving the change, that would be scope creep.

I would first understand the business need and analyze the impact on:

- Requirements
- Timeline
- Cost and resources
- Dependencies
- Technical integrations
- Testing and UAT

Then I would document or refine the change and follow the organization's change-control process or Agile backlog prioritization process.

Scope creep can lead to delays, increased cost, resource pressure, changing priorities, and reduced product quality.

In simple terms:

Scope creep = New work gets added without proper evaluation and approval.

The key difference is:

Approved change = controlled change ✅
Uncontrolled change = scope creep ❌

⭐ Easy interview memory

Remember C-I-A-P:

C → Change requested
I → Impact analyzed
A → Approval/prioritization
P → Properly planned

If these steps are skipped and work simply gets added → Scope Creep.

Likely follow-up: *"How do you prevent scope creep?"*

I prevent scope creep by establishing clear scope and out-of-scope items, maintaining traceability, documenting requirements, analyzing change impacts, communicating with stakeholders, and ensuring changes follow the agreed change-control or backlog-prioritization process.

---

30. What is Gap Analysis?

Gap Analysis compares the current AS-IS state with the desired TO-BE state to identify what needs to change.

What is Gap Analysis?

Gap Analysis is a technique used to compare the current state of a business process or system with the desired future state and identify the gaps between them.

As a Business Analyst, I use Gap Analysis to understand where we are today, where we want to be, what is missing, and what needs to change to reach the target state.

The basic approach is:

1. Current State → Understand how the process or system works today.
2. Future State → Define how the process or system should work.
3. Identify the Gap → Find what is missing, inefficient, or different.
4. Action Plan → Define the changes or requirements needed to close the gap.

For example, in a Healthcare RCM project:

Current State:
Insurance eligibility is verified manually by billing staff, which takes time and can result in errors.

Future State:
The system should automatically verify insurance eligibility through an eligibility API before claim submission.

Gap:

- No automated eligibility integration
- Manual verification process
- No real-time eligibility status
- Limited validation of insurance information

Action Plan:

- Integrate an eligibility API
- Define required data fields and business rules
- Add validation
- Display eligibility status
- Define error and fallback handling
- Test and validate the new process

As a BA, I would document these gaps, analyze their business impact, prioritize them, and convert the required changes into requirements, user stories, process changes, or other appropriate deliverables.

Gap Analysis can be used for process improvement, system implementation, digital transformation, compliance, and moving from a legacy system to a new system.

In simple terms:

Gap Analysis = Where we are → Where we want to be → What is missing → How we close the gap.

Likely follow-up: *"What is the difference between Gap Analysis and Fit-Gap Analysis?"*

Gap Analysis generally identifies differences between the current and desired state. Fit-Gap Analysis is commonly used during system implementation to compare business requirements with what the proposed/standard system can already support, identifying what fits and what requires configuration, customization, or another solution.

Example:

AS-IS

Billing staff manually verifies insurance.

TO-BE

System automatically verifies eligibility through an API.

Gap

No automated eligibility integration.

Then the BA determines:

- Business impact
- Required functionality
- Technology changes
- Dependencies
- Data requirements
- Integration requirements
- Risks

📌 SECTION 5 — SQL

31. I see SQL on your CV. What do you do using SQL?

As a BA, I mainly use SQL for data validation, analysis and troubleshooting rather than database administration.

I use SELECT statements, filtering, joins, aggregate functions, GROUP BY, subqueries and sorting.

For example, if a dashboard shows a certain number of denied claims, I can query the underlying data to validate whether the number is correct.

I can also use SQL to investigate data discrepancies and support QA or UAT.

As a BA, I don't primarily use SQL to develop the application.

I use it to understand the data, validate requirements, investigate issues, and confirm that the system is producing the expected results.

In simple terms:

SQL helps me ask questions from the database and validate whether the data and system behavior match the business requirements.

For example, in a **Healthcare RCM project**, I might use SQL to:

- Verify patient and claim data
- Check whether claims are associated with the correct patient
- Analyze claim status such as submitted, paid, rejected, or denied
- Calculate metrics such as denial rate or total claim amount
- Identify duplicate claims
- Validate data after a system change or integration
- Investigate data-related issues reported by users or QA

---

32. Primary Key vs Foreign Key

Primary Key vs Foreign Key

A Primary Key is a column, or combination of columns, that uniquely identifies each record in a table. It must be unique and cannot contain NULL values.

A Foreign Key is a column that references the Primary Key of another table. It is used to establish a relationship between tables and maintain referential integrity.

For example, in a Healthcare RCM system:

Patient Table

Patient_ID	Patient_Name
101	John
102	Sarah

Here, Patient_ID is the Primary Key because it uniquely identifies each patient.

Claims Table

Claim_ID	Patient_ID	Claim_Amount
C001	101	500
C002	101	750
C003	102	600

Here, Claim_ID is the Primary Key of the Claims table, while Patient_ID is a Foreign Key because it references Patient_ID in the Patient table.

This relationship allows one patient to have multiple claims.

As a BA, understanding primary and foreign keys helps me understand data relationships, validate requirements, write SQL queries, investigate data issues, and verify data during testing or integration.

In simple terms:

Primary Key = uniquely identifies a record.
Foreign Key = connects one table to another.

Example:
Patient.Patient_ID → Primary Key
Claims.Patient_ID → Foreign Key

Likely follow-up: "Can a Foreign Key contain duplicate values?"

Yes. A foreign key can contain duplicate values. For example, one patient can have multiple claims, so the same Patient_ID can appear in multiple rows of the Claims table.

Primary Key

Uniquely identifies a record.

Patient

patient_id

Foreign Key

References another table's primary key.

Claim

claim_id

patient_id

claim.patient_id references patient.patient_id.

---

33. What are SQL joins?

INNER JOIN

Only matching records.

LEFT JOIN

All records from left table + matching records from right.

RIGHT JOIN

All records from right + matching left.

FULL OUTER JOIN

All records from both sides.

For BA interviews, be particularly comfortable with:

INNER JOIN + LEFT JOIN

---

34. DELETE vs TRUNCATE vs DROP

DELETE	TRUNCATE	DROP
Removes rows	Removes all rows	Removes table
WHERE possible	No WHERE	Table structure removed
Table remains	Table remains	Table removed

Memory:

DELETE → rows

TRUNCATE → all rows

DROP → table

---

35. Maximum salary

```
SELECT MAX(salary)
FROM employee;
```

Second highest salary

```
SELECT MAX(salary)
FROM employee
WHERE salary < (
   SELECT MAX(salary)
   FROM employee
);
```

Using DENSE_RANK

```
SELECT salary
FROM (
   SELECT salary,
        DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
   FROM employee
```

) x

WHERE rnk = 2;

📌 SECTION 6 — API

36. What is an API?

API stands for Application Programming Interface. It provides a defined way for one software system to communicate with another software system.

API stands for Application Programming Interface. It is a mechanism that allows two different software systems to communicate with each other and exchange data in a defined way.

For example, in a Healthcare RCM system, the application may need to verify a patient's insurance eligibility. Instead of manually checking the payer's system, the RCM application can communicate with an eligibility API.

The flow could be:

RCM System → API → Payer System
Request: Patient and insurance information
Response: Eligibility status such as Active, Inactive, or an error

From a Business Analyst perspective, I focus on understanding:

- Business purpose – Why is the API required?
- Endpoint – Where is the API accessed?
- HTTP method – GET, POST, PUT, DELETE, etc.
- Request – What data does our system send?
- Response – What data does the external system return?
- Data mapping – How do fields in one system correspond to fields in another?
- Authentication/authorization – How is access secured?
- Business rules and validations – What data is required and what conditions apply?
- Error handling – What happens if the API fails, times out, or returns an error?
- Dependencies – Which external systems or services are involved?

I may also work with developers and QA to clarify API requirements and support API testing using tools such as Postman, depending on the project.

For example, if the eligibility API is unavailable, the requirement may define a retry or fallback mechanism, such as polling or manual follow-up, depending on the business need.

In simple terms:

An API is a bridge that allows one system to communicate with another system and exchange information in a defined format.

Example:
RCM → Eligibility API → Payer → Eligibility Response → RCM

Example:

RCM System

↓

Eligibility API

↓

Insurance System

↓

Response

↓

RCM System

Advantages

- Integration
- Automation
- Data exchange
- Reusability
- Reduced manual effort

- System interoperability

When a client sends an API request, the request goes to the API endpoint, where the server receives and processes it according to the defined rules. The server then sends a response back to the client.

The typical flow is:

1. Client sends a request
The application sends a request to a specific API endpoint using an HTTP method such as GET, POST, PUT, or DELETE.

2. Authentication/Authorization
The API verifies whether the requester is authenticated and has permission to access the resource.

3. Request validation
The server validates the request, such as checking required fields, data format, and business rules.

4. Business processing
The API processes the request and may retrieve or update information in a database or communicate with another system.

5. Response is generated
The server sends a response containing the result, usually with a status code and response data.

For example, in a Healthcare RCM system, when a billing user requests insurance eligibility:

RCM System
↓
POST Eligibility API
↓
Send patient + insurance information
↓
Authentication & validation
↓
Payer processes the request
↓
Response: Active / Inactive / Error

↓

RCM displays the eligibility status

For example, a successful response may return 200 OK along with the eligibility information. If required information is missing, the API may return a client error such as 400 Bad Request. If the external service is unavailable, it may return a server/service-related error.

As a BA, I focus on understanding the business purpose, endpoint, method, request/response fields, validations, business rules, authentication, error handling, data mapping, and dependencies.

In simple terms:

API Request → Authentication → Validation → Processing → Response

The important thing is that an API doesn't just “send data.” It follows a defined contract for what is requested, how it is validated and processed, and what response should be returned.

Likely follow-up: *“What are the main components of an API request?”*

HTTP method, endpoint/URL, headers, authentication information, query/path parameters when applicable, and request body/payload when applicable.

Understand this flow:

Client

↓

Endpoint

↓

Authentication

↓

HTTP Method

↓

Request Headers

↓

Request Body

↓

Server

↓

Response

↓

Status Code

↓

Response Body

Example:

POST

/claims

Request:

```
{
  "patientId": "P1001",
  "insuranceId": "INS5001"
}
```

Response:

```
{
  "claimId": "CLM10001",
  "status": "SUBMITTED"
}
```

---

38. What are HTTP methods?

GET

Retrieve data.

POST

Create/submit data.

PUT

Update/replace a resource.

PATCH

Partially update.

DELETE

Delete resource.

---

39. What are common HTTP status codes?

An HTTP status code tells the client the outcome of an API request.

HTTP status codes are 3-digit codes returned by an API to indicate the result of a request. They help us understand whether the request was successful, had a client-side problem, or encountered a server-side problem.

The most common status codes I would know as a BA are:

2xx – Success

- 200 OK → Request was successful.
- 201 Created → A new resource was successfully created.
- 204 No Content → Request was successful, but there is no response body.

4xx – Client-side Error

- 400 Bad Request → The request is invalid, for example, missing or incorrect data.
- 401 Unauthorized → Authentication is missing or invalid.
- 403 Forbidden → The user is authenticated but does not have permission to access the resource.
- 404 Not Found → The requested endpoint or resource was not found.
- 409 Conflict → The request conflicts with the current state of the resource, such as trying to create a duplicate record.

5xx – Server-side Error

- 500 Internal Server Error → Unexpected error on the server.
- 502 Bad Gateway → A gateway/proxy received an invalid response from another server.
- 503 Service Unavailable → The service is temporarily unavailable, overloaded, or under maintenance.
- 504 Gateway Timeout → The gateway did not receive a response from the upstream server within the expected time.

For example, in a Healthcare RCM eligibility API:

- Valid eligibility request → 200 OK
- Missing insurance ID → 400 Bad Request
- Invalid/expired authentication token → 401 Unauthorized
- User doesn't have permission → 403 Forbidden
- Eligibility service is unavailable → 503 Service Unavailable
- External payer does not respond in time → 504 Gateway Timeout

As a BA, I don't need to memorize every status code, but I need to understand what the response means, what business behavior should happen, and what error or fallback handling is required.

In simple terms:

2xx → Success ✅
4xx → Problem with the request/client ❌
5xx → Problem on the server/service side ⚠️

⭐ Easy interview memory

2 = Success
4 = Client problem
5 = Server problem

The most important ones to remember:

200 → OK
201 → Created
400 → Bad Request

401 → Authentication problem
403 → Permission problem
404 → Not Found
409 → Conflict
500 → Server Error
503 → Service Unavailable
504 → Timeout

Likely follow-up: *"What is the difference between 401 and 403?"*

401 means the request has failed authentication—typically the user/token is missing or invalid.
403 means the requester is authenticated but does not have permission to perform that action.

---

40. REST vs SOAP

REST	**SOAP**
Architectural style	Protocol
Commonly JSON	XML
Lightweight	More formal
HTTP commonly used	Can use multiple transport mechanisms
Flexible	Strict standards
Common modern web APIs	Common in enterprise/legacy integrations

Strong answer:

REST is an architectural style commonly used with HTTP and JSON, while SOAP is a protocol based on XML and defined standards. SOAP generally has stricter contracts and enterprise features, whereas REST is usually simpler and more lightweight.

---

45. Scrum events

Scrum has five events that provide a regular rhythm for the Scrum Team to inspect progress, collaborate, get feedback, and adapt.

The five Scrum events are:

1. Sprint

A Sprint is a fixed time-box of one month or less during which the Scrum Team works toward a Sprint Goal and creates a usable product Increment.

Many teams use a two-week Sprint, although the Scrum framework allows a Sprint of one month or less.

2. Sprint Planning

Sprint Planning happens at the beginning of the Sprint.

The team discusses:

- Why is this Sprint valuable? → Sprint Goal
- What can be done? → Select Product Backlog Items
- How will the selected work be done? → Create the plan

As a BA, I help clarify requirements, user stories, acceptance criteria, business rules, dependencies, and assumptions so the team can understand the work.

3. Daily Scrum

The Daily Scrum is a 15-minute event for Developers, held every working day of the Sprint.

Its purpose is to inspect progress toward the Sprint Goal and adapt the plan if needed.

It is not simply a status meeting for the Scrum Master or BA.

As a BA, I can help explain business functionality, capture feedback, clarify requirements, and translate valid feedback into backlog items.

4. Sprint Review

The Sprint Review happens at the end of the Sprint.

The Scrum Team and stakeholders inspect the Increment and discuss what was accomplished, what has changed, and what should be done next.

Stakeholder feedback can lead to changes or reprioritization of the Product Backlog.

As a BA, I can help explain business functionality, capture feedback, clarify requirements, and translate valid feedback into backlog items.

5. Sprint Retrospective

The Retrospective happens after the Sprint Review and before the next Sprint Planning.

The Scrum Team discusses how the Sprint went, including people, collaboration, process, tools, and quality, and identifies improvements for the next Sprint.

For example:

- What went well?
- What didn't go well?
- What should we improve?

The team then identifies actionable improvements.

Healthcare RCM example

Suppose our Sprint Goal is:

“Enable billing users to verify insurance eligibility.”

Sprint Planning → Select eligibility stories and plan the work.

Daily Scrum → Developers inspect progress toward the Sprint Goal and adjust their plan.

Sprint Review → Demonstrate the eligibility feature to stakeholders and collect feedback.

Retrospective → Discuss what went well and what should improve in the next Sprint.

Sprint → The complete time-box in which all this work happens and a usable Increment is created.

In simple terms:

Sprint = Time-box
Planning = Decide the goal and work
Daily Scrum = Inspect progress and adapt the plan
Review = Inspect product and get feedback
Retrospective = Improve the way of working

Important interview point

Don't say:

❌ “Daily Scrum is where everyone tells the Scrum Master what they did yesterday.”

Better:

✅ “Daily Scrum is a 15-minute event for Developers to inspect progress toward the Sprint Goal and adapt their plan.”

Likely follow-up: *“What is the difference between Sprint Review and Sprint Retrospective?”*

Sprint Review focuses on the product and stakeholder feedback—“Are we building the right thing?” Retrospective focuses on the team's way of working—“How can we work better?”

.

---

48. Epic vs Feature vs User Story

These are different levels of breaking down a large business requirement into smaller, actionable pieces of work.

1. Epic

An Epic is a large body of work or business capability that is too big to complete as a single user story. It is usually broken down into multiple features or user stories.

Example:
Epic: Healthcare Claims Management

This could include claim submission, claim tracking, denial management, and payment posting.

2. Feature

A Feature is a specific capability or functionality that provides value to the user and contributes to an Epic.

Example:
Feature: Insurance Eligibility Verification

The feature can then be broken into smaller user stories.

3. User Story

A User Story describes a specific user need from the user's perspective.

Typical format:

As a [user], I want [functionality], so that [business value].

Example:

As a billing user, I want to verify a patient's insurance eligibility so that I can confirm coverage before submitting a claim.

A user story should have clear acceptance criteria so the team knows what conditions must be satisfied.

4. Task

A Task is a smaller piece of work required to complete a user story. Tasks are generally more implementation-oriented and are created by the team.

For example, for the insurance eligibility story:

- Analyze eligibility API requirements
- Configure eligibility API integration
- Create eligibility screen
- Implement validation
- Handle API timeout/error scenarios
- Write automated/unit tests

Healthcare RCM hierarchy

Epic
→ Claims Management

Feature
→ Insurance Eligibility Verification

User Story
→ As a billing user, I want to verify insurance eligibility so that I can confirm coverage before submitting a claim.

Tasks
→ API analysis
→ UI development
→ Validation
→ Error handling
→ Testing

As a BA, I typically focus more on the business need, features, user stories, acceptance criteria, business rules, and dependencies, while working with the team to clarify the smaller implementation tasks.

In simple terms:

Epic = Big business area
Feature = Specific capability
User Story = User need/value
Task = Work needed to complete the story

Hierarchy:

Epic → Feature → User Story → Task

Likely follow-up

Q: Is the hierarchy always exactly Epic → Feature → User Story → Task?

Not necessarily. The hierarchy depends on the organization's Agile tool and process. Some teams use Epic → Story → Task, while others use Epic → Feature → User Story → Task. The important concept is progressive decomposition from a large business objective into smaller, actionable work.

Example:

Epic: Claims Management

Feature: Claim Submission

User Story: Submit claim electronically

Task: Develop claim submission API integration

📌 SECTION 8 — JIRA

These are very likely questions for a BA position.

49. What do you do in Jira?

I use Jira to manage and track Agile work. As a BA, I can create and refine epics, features and user stories, write acceptance criteria, add business details, prioritize or support backlog refinement, track dependencies and monitor story progress.

“I use Jira to manage and track requirements and the Agile delivery process.

As a Business Analyst, I mainly work on creating and refining **Epics, Features, and User Stories**, writing clear **Acceptance Criteria**, and maintaining the **Product Backlog**.

I also participate in **backlog refinement and Sprint Planning**, clarify requirements and business rules for developers and testers, and track the progress of stories through the Jira workflow.

For example, in a healthcare RCM project, I might create a user story for **insurance eligibility verification**, add the acceptance criteria and business rules, link related tasks or defects, and track the story from **To Do → In Progress → Testing → Done**.

I also use Jira to track dependencies, defects, requirement changes, and Sprint progress, so that the team and stakeholders have visibility into the work.”**

---

50. What is backlog refinement?

Backlog refinement is a collaborative activity where upcoming backlog items are reviewed, clarified, split if necessary, estimated and prepared for future Sprints.

“Backlog refinement is a regular Scrum activity where the team reviews and prepares upcoming Product Backlog items so they are clear, understood, and ready for future Sprints.

As a Business Analyst, I usually help by **clarifying the business requirement, breaking down larger items into user stories, writing or refining acceptance criteria, identifying business rules, dependencies, and gaps, and answering questions from developers and testers.**

For example, in a healthcare RCM project, if we have a backlog item called **‘Insurance Eligibility Verification’**, during refinement we may discuss the user story, required patient and insurance information, validation rules, API dependencies, error scenarios, and acceptance criteria.

The goal is to make sure the team has a **shared understanding of the requirement** before it is considered for Sprint Planning.

So, in simple terms, **backlog refinement means reviewing, clarifying, splitting, and preparing backlog items for future Sprints.”**

50a Who participates in backlog refinement?

"Backlog refinement is usually a collaborative activity involving the Product Owner, Business Analyst, Developers, and QA or Testers.

The **Product Owner** provides business priorities and clarifies the product value.

As a **Business Analyst**, I help clarify requirements, refine user stories, define acceptance criteria, identify business rules, dependencies, and gaps.

Developers provide technical input, identify technical dependencies or constraints, and help determine whether the story is clear enough to implement.

QA/Testers review the requirements from a testing perspective and identify missing scenarios or acceptance criteria.

The goal is to make sure everyone has a **common understanding of the backlog items before they are considered for Sprint Planning**.

So, in simple terms: **PO gives priority, BA clarifies requirements, Developers assess implementation, and QA validates testability.**"

---

51. What is Definition of Ready?

"Definition of Ready, or DoR, is a set of criteria that a user story should meet before the team takes it into a Sprint.

It helps the team make sure the story is **clear, understood, testable, and feasible** before Sprint Planning.

As a Business Analyst, I help ensure that the story has a clear business requirement, proper acceptance criteria, required business rules, dependencies and assumptions are identified, and any necessary designs or API details are available.

For example, for a healthcare RCM story like **'Verify Insurance Eligibility'**, the team should know what information is required, what the expected response should be, what validation rules apply, and what should happen in error scenarios before taking the story into the Sprint.

So, in simple terms, **Definition of Ready means: Is this user story clear and prepared enough for the team to work on it?**"

A story is sufficiently prepared to enter development.

Typical conditions:

- Requirement understood
- Acceptance criteria available
- Dependencies identified
- Designs available if needed
- Questions resolved

---

52. What is Definition of Done?

"Definition of Done, or DoD, is a shared checklist of conditions that a user story must meet before the team considers it completely finished.

It ensures that the work is not just developed, but also **tested, validated, and meets the required quality standards**.

For example, in a healthcare RCM project, if we have a story for **Insurance Eligibility Verification**, the Definition of Done might include: development completed, acceptance criteria satisfied, testing completed, defects resolved, business rules validated, and the story accepted by the Product Owner.

As a Business Analyst, I help make sure the **requirements and acceptance criteria are clear and properly validated**.

So, in simple terms, **Definition of Done answers: "Is this work truly complete and ready to be considered finished?""**

DoR: Is the story **ready to start?**
DoD: Is the story **complete to finish?**

A shared agreement about when work is considered complete.

Could include:

- Development completed
- Code reviewed
- Testing completed

- Acceptance criteria passed
- No critical defects
- Documentation updated
- UAT completed where applicable

📌 SECTION 9 — BEHAVIORAL / SCENARIO QUESTIONS

53. Rate yourself 1–10 in SQL/API/Power BI

For your current profile, don't overrate yourself.

Example:

I would rate myself around 7 out of 10 in SQL from a BA perspective. I'm comfortable with querying, filtering, joins, aggregation and data validation.

For APIs, I would rate myself around 7 because I understand REST, request-response flow, JSON, authentication concepts, status codes and integration scenarios.

For Power BI, I would rate myself around 6. I can work with datasets, relationships, visuals, filters and business dashboards, while continuing to improve advanced DAX and modelling.

Then say:

"I prefer to rate myself based on practical BA usage rather than pure technical depth."

That's a mature answer.

---

54. How do you handle changing requirements?

Use:

Understand → Impact → Prioritize → Approve → Update → Communicate

First, I understand why the requirement changed and what business value it provides.

Then I perform impact analysis covering scope, timeline, development, testing, dependencies and risks.

I discuss the impact with the Product Owner and stakeholders and prioritize the change.

If approved, I update the relevant requirements, user stories and acceptance criteria and communicate the change to the affected teams.

---

55. Last-minute scope change?

Don't immediately say yes.

I would first understand the urgency and business reason. Then I would assess the impact on the current Sprint or release. If the change is critical, I would discuss whether lower-priority work can be removed or whether the change should be planned for the next Sprint. The important point is to make the impact visible before committing.

---

56. Project is delayed but business wants faster delivery

Excellent framework:

Assess → Identify bottleneck → Prioritize → Re-plan → Communicate

First, I would identify the reason for the delay—requirements, development, dependencies, resources, testing or scope.

Then I would identify the critical items required for business value and discuss options with the Product Owner and technical team.

We could prioritize an MVP, defer lower-priority features, parallelize activities where possible or address critical dependencies.

I would communicate a realistic delivery plan rather than promising an unrealistic date.

---

57. How do you handle stakeholder conflict?

First, I listen to both stakeholders and understand the reason behind their positions.

I bring the discussion back to the business objective, user impact and available data rather than personal opinions.

I document the different viewpoints, identify the conflict and facilitate a discussion to reach agreement.

If the decision requires business ownership, I escalate it to the appropriate Product Owner or decision-maker rather than making an unauthorized decision myself.

---

58. What if a stakeholder doesn't know what they want?

Very common.

I don't simply ask, "What requirement do you want?"

I start with the business problem. I ask what they are trying to achieve, how the current process works, what problems they face and what outcome they expect.

I use the AS-IS process, examples, prototypes or workshops to help them articulate the requirement.

Then I validate the proposed TO-BE process with them.

---

59. Developer says requirement is technically impossible. What do you do?

I first understand the technical limitation rather than immediately pushing the original requirement.

Then I go back to the business objective and determine whether there are alternative ways to achieve the same outcome.

I discuss the alternatives with the technical team and stakeholders and document the agreed solution and its limitations.

This is a **very strong BA answer**.

---

60. QA says requirement is unclear. What do you do?

I review the requirement and acceptance criteria with QA to understand the ambiguity.

If necessary, I discuss the expected behavior with the stakeholder or Product Owner.

Once clarified, I update the requirement and acceptance criteria and communicate the clarification to both development and QA so everyone works from the same understanding.

---

61. How do you prioritize requirements?

Know:

MoSCoW

Must Have

Essential.

Should Have

Important but not absolutely critical.

Could Have

Nice to have.

Won't Have

Not planned for current release.

You can also mention:

- Business value
- Customer impact
- Risk
- Regulatory importance
- Dependency
- Cost/effort
- Urgency

For healthcare, **regulatory/compliance requirements can become very high priority**.

📌 SECTION 10 — UAT & PRODUCTION

62. What is UAT?

UAT stands for User Acceptance Testing. It is the stage where business users validate whether the solution meets their business needs and is acceptable for business use.

“UAT stands for User Acceptance Testing. It is the testing performed by business users or end users to confirm that the system meets the business requirements and is ready for business use.

As a Business Analyst, I help prepare or clarify **UAT scenarios and acceptance criteria**, coordinate with business users, support them during testing, document issues or feedback, and make sure the requirements are properly validated.

For example, in a healthcare RCM system, if we develop **Insurance Eligibility Verification**, the business user would test scenarios such as valid insurance, inactive insurance, missing information, and payer response errors to confirm that the system behaves as expected.

If all critical scenarios pass and the business is satisfied, the functionality can receive **business acceptance/sign-off**.

So, in simple terms, **UAT answers: "Does the system work correctly for the business and meet the actual business need?"**"

BA role:

"As a Business Analyst, my role in UAT is mainly to make sure the business requirements are properly validated by the business users.

Before UAT, I help prepare UAT scenarios and test cases based on the requirements and acceptance criteria, make sure the required test data is available, and coordinate with the business users and QA team.

During UAT, I help users understand the requirements and business rules, clarify any questions, document issues or gaps they identify, and coordinate with the development and QA teams for resolution.

After fixes, I help with retesting and tracking UAT results, and support the business users in confirming whether the functionality is acceptable.

For example, in a healthcare RCM system, for Insurance Eligibility Verification, I would help define scenarios such as active insurance, inactive insurance, missing information, and payer errors. Business users would execute or validate these scenarios, while I would support them and track any issues.

So, simply, BA = Prepare → Coordinate → Clarify → Track → Validate → Support business acceptance."

- Prepare/support UAT scenarios

- Coordinate users
- Explain requirements
- Clarify expected behavior
- Track defects
- Collect feedback
- Support sign-off

SIT	UAT
System Integration Testing	User Acceptance Testing
Integration/system focused	Business/user focused
Checks data flow & interfaces	Checks business requirements
Usually QA/technical team	Business/end users
“Does it integrate correctly?”	“Does it meet our business need?”

---

63. UAT vs QA testing?

“QA testing and UAT have different purposes.

QA testing is performed by the testing team to verify that the system works correctly according to the technical and functional requirements. QA focuses on things like functional testing, integration, regression, negative scenarios, and defects.

UAT, or User Acceptance Testing, is performed by business users or end users to confirm that the system supports the actual business process and meets the business requirements.

For example, in a healthcare RCM system, QA might test whether the Insurance Eligibility API correctly handles valid, invalid, and error responses.

During UAT, a billing user would test the complete business scenario—such as verifying a patient's insurance before submitting a claim—and confirm that the functionality works for their actual business process.

As a BA, I support both by clarifying requirements and acceptance criteria, but QA validates system quality, while UAT validates business acceptance.

So, simply:

QA = Does the system work correctly?
UAT = Does the system meet the business need?”

QA → Quality & defects
UAT → Business acceptance

QA	**UAT**
Technical/quality validation	Business acceptance
QA team	Business/end users
Finds defects	Determines business suitability
System behavior	Business outcome

63a is SIT and QA testing different

“Yes, they are different, but they are related. QA is the broader quality assurance and testing process, while SIT is a specific type of testing focused on integration between systems or modules.

For example, in a healthcare RCM system, QA may cover functional, regression, negative, and integration testing. During SIT, the team specifically verifies whether the RCM system correctly communicates with an external payer API and whether the request, response, and data flow work correctly.

So I would say: QA is the broader testing function, and SIT is one type of testing performed as part of that overall process.”

---

64. What happens after UAT?

“After UAT, the next steps depend on the UAT outcome.

If the business users are satisfied and all critical scenarios pass, the business provides **UAT sign-off or acceptance**.

After that, the team prepares for **production deployment**. This may include final regression checks, deployment planning, release communication, data migration if required, and production readiness activities.

If UAT identifies defects or gaps, those issues are documented, prioritized, fixed by the development team, and then retested before final acceptance.

As a BA, I help track UAT results, clarify requirements, coordinate with business users, support issue resolution, and make sure the agreed requirements are properly accepted.

So, simply:

UAT → Fix/Retest if needed → Business Sign-off → Production Readiness → Go-Live → Post-Production Support.”

UAT

↓

Defect Fixes

↓

Retesting

↓

Business Sign-off

↓

Release Approval

↓

Deployment

↓

Production Validation

↓

Monitoring / Feedback

64a What is the BA's role after go-live?"

"After go-live, the BA continues to support the business and the product to make sure the solution is working as expected.

I would monitor business feedback, production issues, defects, and any gaps between the expected and actual business process. If users report an issue, I would help understand the business impact, clarify the requirement, and coordinate with the development and QA teams for resolution.

I would also collect enhancement requests and new business requirements, perform impact analysis, and work with the Product Owner to prioritize them in the backlog.

For example, after launching an RCM eligibility feature, if billing users report that some payer responses are not being displayed correctly, I would analyze the issue from the business perspective, clarify the expected behavior, work with the technical team, and support validation of the fix.

So, in simple terms, after go-live, the BA helps stabilize the solution, resolve business issues, collect feedback, and identify improvements for future releases."

64b What is the difference between Hypercare and normal production support?

"The main difference is the intensity and timing of the support.

Hypercare is temporary, intensive support immediately after go-live. The team closely monitors the system, quickly addresses critical production issues, and works closely with users to stabilize the solution.

Normal production support starts after the system is stable. It is an ongoing support process where incidents, service requests, defects, and enhancement requests are handled according to the organization's normal support process and priorities.

For example, after an RCM system goes live, during hypercare we may closely monitor claim submission and eligibility issues every day and prioritize critical problems immediately.

Once the system is stable, those issues move into the normal production support process with standard **SLA, ticketing, prioritization, and release procedures**.

So, simply:

Hypercare = Temporary + Intensive + Post-go-live stabilization
Production Support = Ongoing + Standard + Operational support."

📌 SECTION 11 — BUSINESS PROCESS

65. What is AS-IS and TO-BE?

"AS-IS and TO-BE are used to understand the current business process and define the desired future process.

AS-IS describes how the process works today, including the current steps, systems, people, problems, and limitations.

TO-BE describes how we want the process to work after the improvement or new solution is implemented.

For example, in a healthcare RCM process, the AS-IS process might involve billing staff manually checking insurance eligibility through different payer portals.

The TO-BE process could be an RCM system that automatically sends the patient's insurance information through an eligibility API and displays the eligibility status in the system.

As a BA, I analyze the AS-IS process, identify pain points and gaps, and work with stakeholders to design and document the TO-BE process.

So, simply:

AS-IS = How things work today
TO-BE = How we want things to work in the future."

65a How do you identify the gap between AS-IS and TO-BE?"

"To identify the gap between AS-IS and TO-BE, I compare the current process with the desired future process and identify what is missing, inefficient, or needs to change.

First, I understand and document the AS-IS process by talking to stakeholders, reviewing existing documentation, and observing the current workflow.

Then I define the TO-BE process based on business goals, stakeholder expectations, and the proposed solution.

After that, I compare both processes step by step and identify gaps in areas such as process steps, people, systems, data, business rules, integrations, and controls.

For example, in healthcare RCM, the AS-IS process may require staff to manually verify insurance through payer portals, while the TO-BE process uses an eligibility API. The gaps could include API integration, data mapping, validation rules, error handling, and user training.

Finally, I document the gaps, analyze their impact, prioritize them, and convert the required changes into requirements, user stories, or process improvements.

So, simply:

AS-IS → TO-BE → Compare → Identify Gaps → Analyze Impact → Define Actions."

---

66. What is process mapping?

Process mapping visually represents the sequence of activities, decisions, actors and system interactions in a business process.

"Process mapping is a visual representation of how a business process works from start to finish. It shows the sequence of activities, decisions, people or systems involved, inputs, outputs, and handoffs.

As a Business Analyst, I use process mapping to understand the **AS-IS process**, identify bottlenecks, unnecessary steps, gaps, and dependencies, and then help design the **TO-BE process**.

For example, in a healthcare RCM process, I might map:

Patient Registration → Insurance Verification → Claim Creation → Claim Validation → Claim Submission → Payer Response → Payment/Denial Handling.

While mapping the process, I may identify that insurance verification is done manually, which creates delays. That can become an opportunity for improvement in the TO-BE process, such as automated eligibility verification through an API.

I can create process maps using tools like **Visio, Lucidchart, Draw.io, or Jira/Confluence diagrams**, depending on the organization.

So, simply, **process mapping means visually documenting how a process works so we can understand it, analyze it, and improve it.**"

Common formats:

- Flowchart
- BPMN
- UML Activity Diagram
- Swimlane diagram

66a What is the difference between Process Mapping and BPMN?

"Process mapping and BPMN are related, but they are not exactly the same.

Process mapping is a general approach to visually represent a business process. It can be as simple as showing activities and decisions using basic flowchart symbols.

BPMN, or Business Process Model and Notation, is a standardized notation for modeling business processes. It provides specific symbols and rules for representing events, activities, gateways, messages, participants, and process flows.

For example, in a healthcare RCM process, a simple process map could show:

Patient Registration → Insurance Verification → Claim Submission → Payment

In BPMN, we could represent the same process in more detail, including different pools or lanes for Billing Staff and Payer, decision gateways for claim validation, events, and message flows between systems.

As a BA, I would use a simple process map when I need to communicate a process quickly, and BPMN when I need a standardized and more detailed representation, especially for complex processes involving multiple teams or systems.

So, simply:

Process Mapping = General visual representation of a process
BPMN = Standardized notation for detailed process modeling."

Easy to remember

Process Map = What happens?
BPMN = What happens + who + when + decisions + messages + system interactions

Interview tip: Don't say BPMN and process mapping are completely different. BPMN is one standardized way of creating process models/maps.

---

67. BPMN vs UML

"BPMN and UML are both used for visual modeling, but they have different purposes.

BPMN, or Business Process Model and Notation, is mainly used to model business processes and workflows. It focuses on activities, events, decisions, roles, handoffs, and the flow of work between people and systems.

UML, or Unified Modeling Language, is a broader modeling language used mainly for software and system analysis and design. It includes different diagrams such as Use Case, Activity, Sequence, Class, and State diagrams.

For example, in a healthcare RCM system, I might use BPMN to show the complete claim process from claim creation to payer response, including billing staff and payer interactions.

I might use UML Use Case to show what different users can do in the system, or a UML Sequence Diagram to show how the RCM system, API, and payer communicate step by step.

So, simply:

BPMN = Business process and workflow modeling
UML = System/software modeling using different types of diagrams."

BPMN: How does the business process flow?
UML: How does the system behave and interact?

Interview tip: UML is not only for developers. As a BA, you commonly use Use Case, Activity, and Sequence diagrams to clarify requirements and system behavior.

BPMN

Primarily focused on **business processes and workflows**.

UML

Used to model **software/system behavior and structure**.

Examples:

BPMN → Claims processing workflow

UML Use Case → Billing executive submits claim

UML Activity → Claim submission activities

📌 SECTION 13 — POWER BI

69. What do you do in Power BI?

From a BA perspective, I use Power BI to convert business data into meaningful dashboards and reports.

I first understand what business decisions the report needs to support and identify the required KPIs.

Then I identify the data sources, transform the data where necessary, create relationships and data models, create measures, build visuals and apply filters or slicers.

Finally, I validate the report numbers against the source data and business requirements.

---

70. Explain how you build a Power BI report

Remember:

Business Requirement

↓

Identify KPIs

↓

Identify Data Sources

↓

Power Query / Data Cleaning

↓

Data Model

↓

Relationships

↓

DAX Measures

↓

Visualizations

↓

Filters / Slicers

↓

Validate

↓

Publish

Healthcare RCM example:

- Claim volume
- Claim acceptance rate
- Denial rate
- Payment amount
- Outstanding AR
- Payer performance
- Denial reasons
- Collection trends

🧠 **THE MOST IMPORTANT 20 TO MASTER FIRST**

Don't try to memorize all 135 immediately.

I'd prioritize these:

Priority	Question
⭐⭐⭐	Tell me about yourself
⭐⭐⭐	Roles & responsibilities
⭐⭐⭐	Day-to-day BA activities
⭐⭐⭐	Requirement gathering
⭐⭐⭐	Stakeholder meeting
⭐⭐⭐	Project explanation
⭐⭐⭐	BA contribution
⭐⭐⭐	Project challenge
⭐⭐⭐	BRD vs FRD
⭐⭐⭐	FRD
⭐⭐⭐	User Story
⭐⭐⭐	Acceptance Criteria
⭐⭐⭐	Functional vs NFR
⭐⭐⭐	API
⭐⭐⭐	REST vs SOAP
⭐⭐⭐	SQL
⭐⭐⭐	Agile/Scrum
⭐⭐⭐	Changing requirements
⭐⭐⭐	UAT

Priority	Question
⭐ ⭐ ⭐	Gap Analysis

🎯 One More Important Thing: Don't Memorize These Answers

This is the part I want you to understand before we start the mock interview.

An interviewer may ask:

"What is gap analysis?"

That's easy.

But then they may immediately ask:

"Okay, give me an example from your project."

Then:

"How did you identify the gap?"

Then:

"Who approved the TO-BE process?"

Then:

"How did you convert that gap into requirements?"

Then:

"Did you document it in BRD or FRD?"

Then:

"How did you validate that requirement?"

So our mock interview will deliberately work this way.

Example interview chain

Interviewer:

What is Gap Analysis?

↓

Give me an example.

↓

What was your AS-IS process?

↓

What was your TO-BE process?

↓

How did you identify the gap?

↓

What requirement came from that gap?

↓

How did you document it?

↓

What user story did you create?

↓

What acceptance criteria?

↓

How did QA test it?

↓

How did you perform UAT?

↓

What happened after deployment?

This is how we will practice.

And I will also deliberately challenge answers that sound like **"textbook BA answers"** so that you learn to respond like a real BA rather than reciting definitions.

📌 SECTION B — AGILE

88. User Story vs Use Case

User Story	Use Case
Common in Agile	Often used for detailed interaction analysis
Short	More detailed
User need/value focused	Actor-system interaction
"As a... I want... so that..."	Actor, preconditions, flow, exceptions

Interview answer

A user story expresses a user need and business value in a concise Agile format. A use case provides more detailed interaction between an actor and the system, including scenarios, preconditions, main flow and exceptions.

---

90. What happens during Sprint Planning?

During Sprint Planning, the Scrum Team discusses what can be delivered during the Sprint and how the work will be accomplished.

The Product Owner provides context around priorities and the Product Goal. The Developers select work based on priority, capacity and their understanding of the work and establish a Sprint Goal.

The team then creates the Sprint Backlog and develops a plan for delivering the selected work.

---

91. What happens during Sprint Review?

Sprint Review is held at the end of the Sprint to inspect the Increment and discuss progress with stakeholders.

The team demonstrates the completed work, receives feedback and discusses what should happen next.

The feedback may lead to changes or additions to the Product Backlog.

Important:

Review = Product

Retrospective = Process/team

---

92. What happens during Retrospective?

The team reflects on how the Sprint went and discusses what went well, what didn't go well and what improvements should be made.

The team identifies actionable improvement items for future Sprints.

Example:

QA was receiving requirements too late.

Improvement:

BA and QA will review acceptance criteria during refinement.

---

93. What is Backlog Refinement?

Backlog refinement is an ongoing activity where upcoming Product Backlog Items are reviewed, clarified, split when necessary, estimated and prepared for future Sprints.

BA activities can include:

- Clarifying requirements
- Updating acceptance criteria
- Splitting stories
- Identifying dependencies
- Answering developer/QA questions

---

94. What is Velocity?

Velocity is a measure of the amount of work a Scrum Team completes during a Sprint, commonly expressed using story points.

Example:

Sprint 1 = 25 points

Sprint 2 = 28 points

Sprint 3 = 27 points

Average velocity ≈ 27 points.

It can help with **forecasting**, but it should not be treated as a performance score.

---

95. What are Story Points?

Story points are a relative estimation technique used by Agile teams to estimate the size or complexity of a Product Backlog Item.

They consider things such as:

- Complexity
- Effort
- Uncertainty
- Dependencies

Example:

Simple story = 2

Medium story = 5

Complex story = 8

Important:

Story points are **not hours**.

---

96. Who Prioritizes the Product Backlog?

The **Product Owner** is accountable for ordering/prioritizing the Product Backlog.

BA's role

A BA can provide:

- Requirement analysis
- Business impact
- Stakeholder input
- Dependencies

- Risk information
- Data
- Acceptance criteria

But don't say:

"The BA owns the Product Backlog priority."

In Scrum, the **Product Owner is accountable** for Product Backlog ordering.

---

97. What is Sprint Goal?

Sprint Goal is the single objective for the Sprint. It provides the team with a common purpose and helps guide decisions during the Sprint.

Example:

"Enable billing users to submit and track insurance claims electronically."

---

98. What is Product Goal?

Product Goal describes a future objective for the product and provides a longer-term target for the Scrum Team.

Example:

"Build an integrated healthcare RCM platform that improves claim processing and revenue visibility."

Difference

Product Goal = longer-term product objective

Sprint Goal = current Sprint objective

📌 SECTION C — BA SCENARIO QUESTIONS

These are extremely important because experienced interviewers often spend more time here than definitions.

---

99. Two stakeholders give conflicting requirements. What do you do?

First, I understand the reason behind each stakeholder's requirement rather than deciding immediately who is right.

I compare both requirements against the business objective, user impact, process, risk and available data.

I facilitate a discussion between the stakeholders and try to reach a common understanding.

If the conflict requires a business decision, I involve the Product Owner or appropriate decision-maker.

Once the decision is made, I document the agreed requirement and communicate it to the affected teams.

---

100. Stakeholder doesn't attend requirement meeting. What do you do?

I would first determine whether that stakeholder is critical for the decision.

If their input is required, I would reschedule or arrange an alternative session.

I wouldn't finalize important requirements based on assumptions without the appropriate stakeholder's input.

If the delay affects the project timeline, I would communicate the dependency and escalate appropriately.

---

101. Developer misunderstood the requirement. What do you do?

First, I understand what the developer interpreted and compare it with the approved requirement and acceptance criteria.

If my requirement was ambiguous, I take responsibility for clarifying it and update the documentation.

If the requirement was already clear but was misunderstood, I explain the expected behavior and ensure the developer and QA team have the same understanding.

I also check whether the misunderstanding has affected other work.

---

102. QA finds a defect that isn't mentioned in the requirement. What do you do?

Don't automatically say it's a defect.

First, I would understand the issue and determine the expected business behavior.

I would review the requirement, acceptance criteria, business rules and stakeholder expectations.

If the behavior violates an existing business requirement, it should be treated as a defect.

If the functionality was never defined and is actually a new business expectation, I would treat it as a requirement or enhancement and follow the appropriate change process.

Excellent BA answer.

---

103. Business wants a feature but there is no budget.

I would understand the business value and urgency first.

Then I would work with the Product Owner and technical team to assess alternatives, effort, impact and possible scope trade-offs.

We could consider an MVP or lower-cost solution.

If there is still no budget, I would document the requirement and prioritize it for a future release rather than committing to something that cannot realistically be delivered.

---

104. Stakeholder asks for a change during UAT.

First, I determine whether the request is a defect, clarification or genuinely new functionality.

If it is a defect against the approved requirement, it should be addressed through defect management.

If it is new functionality, I assess its impact on scope, timeline, testing and release and discuss it with the Product Owner and stakeholders.

Based on priority and impact, it can either be included in the current release if appropriate or moved to the backlog for a future release.

---

105. Production issue reported by business. What do you do?

First, I understand and document the issue, including what the user expected, what actually happened, affected users, business impact and severity.

I coordinate with the technical and support teams to investigate the issue.

I help determine whether it is a defect, data issue, configuration issue or expected behavior.

If necessary, I support root cause analysis and stakeholder communication.

After resolution, I help validate the fix and identify whether any requirement, test case or process needs to be updated to prevent recurrence.

---

106. How do you manage requirement ambiguity?

I don't make assumptions silently.

I identify the ambiguous part and ask targeted questions using examples and scenarios.

I may use process flows, wireframes, prototypes or sample data to make the discussion clearer.

Once the expected behavior is agreed, I update the requirement and acceptance criteria and communicate the clarification to the relevant teams.

---

107. How do you communicate bad news to stakeholders?

I communicate it early rather than waiting until the problem becomes bigger.

I explain the issue objectively, the business impact, the reason, and the available options.

Instead of only saying "we can't do it", I try to provide alternatives and their trade-offs.

I also make sure the stakeholder understands the impact on scope, timeline, cost or quality before a decision is made.

---

108. How do you handle an aggressive stakeholder?

I remain calm and professional and avoid responding emotionally.

I try to understand the underlying concern because sometimes frustration comes from business pressure or unmet expectations.

I bring the discussion back to facts, requirements, business objectives and available options.

If the behavior becomes inappropriate or prevents productive discussion, I involve the appropriate manager or Product Owner while maintaining professionalism.

📌 SECTION D — TECHNICAL BA

109. What is Database Normalization?

Database normalization is the process of organizing data into related tables to reduce unnecessary duplication and improve data integrity.

Simple example

Instead of storing:

Patient Name

Patient Address

Patient Insurance

Patient Insurance

Patient Insurance

repeatedly in every claim record, we can separate patient and insurance information into appropriate related tables.

BA-level understanding

You don't need to be a database architect.

Remember:

Normalization → reduce redundancy + improve data integrity.

---

110. What is a Database?

A database is an organized collection of data that allows applications and users to store, retrieve, update and manage information efficiently.

Example:

Healthcare RCM database could contain:

- Patient
- Insurance
- Claim
- Payment
- Denial
- Provider

Database → Container
Table → Stores a specific category of data
Row → One record
Column → One attribute/field

110a Likely follow-up: "What is a DBMS?"

"DBMS stands for Database Management System. It is software used to create, store, manage, retrieve, and control data in a database.

A DBMS allows users and applications to interact with the database without directly managing how the data is physically stored.

For example, in a healthcare RCM system, a DBMS can manage **patient, claim, insurance, payment, and denial data**. The application can use SQL through the DBMS to retrieve or update that information.

Common examples of DBMS are **MySQL, PostgreSQL, Oracle Database, Microsoft SQL Server, and MongoDB**.

As a Business Analyst, I don't necessarily administer the DBMS, but I need to understand how data is stored, related, accessed, and validated so I can work effectively with developers, testers, and data teams.

So, simply, **Database is where the data is stored, while DBMS is the software that manages and provides access to that data.**"

110b What is the difference between DBMS and RDBMS?

"DBMS and RDBMS are both used to manage databases, but the main difference is how they organize and relate data.

DBMS, or Database Management System, is a general system used to store, manage, and retrieve data. It may support different ways of organizing data.

RDBMS, or Relational Database Management System, stores data in tables and allows those tables to be related to each other using keys such as Primary Keys and Foreign Keys. It generally follows relational rules and commonly uses SQL.

For example, in a healthcare RCM RDBMS, we could have separate Patient, Claims, Insurance, and Payment tables, with relationships between them.

Common RDBMS examples are MySQL, PostgreSQL, Oracle Database, and Microsoft SQL Server.

So, simply:

DBMS = General database management system
RDBMS = Relational database management system using related tables."

Interview tip: You may hear the simplified statement "RDBMS is a type of DBMS." That's a good way to remember the relationship.

---

111. What is an API Endpoint?

An API endpoint is a specific URL or address through which a client can access a particular API resource or operation.

Example:

GET /patients/123

Here:

- GET = HTTP method
- /patients/123 = endpoint/path
- 123 = patient identifier

---

112. What is JSON?

JSON = JavaScript Object Notation

JSON is a lightweight data format commonly used to exchange structured data between systems, especially in REST APIs.

Example:

```
{
  "patientId": "P1001",
  "status": "ACTIVE"
}
```

---

113. Authentication vs Authorization

Authentication and Authorization are related to security, but they answer two different questions.

Authentication verifies who the user is. For example, when I log into a healthcare RCM application using my username, password, and possibly an OTP, the system authenticates me.

Authorization determines what that authenticated user is allowed to access or do. For example, a billing user may be allowed to view and submit claims, while an administrator may also be allowed to manage users and system settings.

So, the sequence is usually:

Authentication → Who are you?
Authorization → What are you allowed to do?

As a BA, I need to define or clarify requirements such as user roles, access permissions, authentication method, restricted data, and what actions each role can perform.

So, simply: Authentication = Identity, Authorization = Permission.”

---

115. What is API Request and Response?

“An API request is the information that one system sends to another system to ask it to perform an action or provide data. An API response is the information that the receiving system sends back as the result of that request.

An API request can contain things like the HTTP method, endpoint, headers, authentication details, query parameters, and request body or payload.

The response usually contains an HTTP status code, response headers, and response body or data.

For example, in a healthcare RCM system, the RCM application may send an eligibility request to a payer API with patient and insurance information.

The payer processes the request and sends a response such as Active, Inactive, or an error message, along with the relevant details.

As a BA, I focus on understanding the business purpose, required request fields, response fields, data mapping, validation rules, authentication, and error scenarios.

So, simply:

API Request = What we send
API Response = What we receive.”

API Request

Client → Server

The client sends a request to the server/API.

Component	Meaning	Example
URL	Where to send the request	/eligibility
Method	What action to perform	POST
Headers	Additional information about the request	Content-Type: application/json

Component	Meaning	Example
Parameters	Extra values used to identify/filter information	patientId=101
Body	Main data being sent	Patient + insurance details

For example:

POST /eligibility

Request body:

```
{
  "patientId": "P1001",
  "insuranceId": "INS456"
}
```

API Response

Server → Client

The server processes the request and sends the result back.

Component	Meaning	Example
Status Code	Result of the request	200 OK
Headers	Information about the response	Content-Type: application/json
Response Body	Actual result/data	Eligibility = Active
Error Information	Details when something goes wrong	Invalid insurance ID

Example response:

```
{
  "patientId": "P1001",
  "eligibilityStatus": "Active"
}
```

with:

200 OK

🔥 *Very important distinction for interviews*

Don't mix these up:

Request

URL + Method + Headers + Parameters + Body

Response

Status Code + Headers + Response Body + Error Information

And remember:

Parameters and Body are not the same thing.

- Parameters → usually pass additional values through the URL, such as query/path parameters.
- Body → carries the main payload/data sent to the server, commonly with POST/PUT/PATCH.

Easy interview formula

Request = Where + What + Extra Information + Data

Response = Result + Information + Data/Error

Think of this as the complete journey of an API call:

Request → Endpoint + Method + Data → API → Response → Status + Data

1. Request — “I want something from another system”

One application wants to communicate with another application.

For example, an RCM system wants to check a patient's insurance eligibility.

So it creates an API request.

2. Endpoint — “Where should I send it?”

The endpoint tells the system where the request should go.

Example:

/eligibility

Think of it like a destination/address.

RCM → /eligibility

3. Method — “What do I want to do?”

The HTTP method tells the API what operation is being requested.

Common methods:

- GET → Retrieve data
- POST → Send/create data
- PUT/PATCH → Update data
- DELETE → Delete data

For our eligibility example, we might use:

POST → I am sending patient/insurance information for verification.

4. Data — “What information am I sending?”

The request may contain data in the request body/payload.

For example:

```
{
  "patientId": "P1001",
  "insuranceId": "INS456",
  "dateOfBirth": "1985-05-10"
}
```

This is the information the payer API needs to process the eligibility request.

5. API — “The middleman/bridge”

The API receives the request and processes it.

It may:

Validate → Authenticate → Apply business rules → Communicate with database/external system → Prepare response

So:

RCM → API → Payer System

6. Response — “Here is the result”

After processing the request, the API sends something back.

For example:


```
{
  "patientId": "P1001",
  "eligibilityStatus": "Active",
  "payer": "ABC Health"
}
```


This is the API response.

7. Status — “Was the request successful?”

The response also normally contains an HTTP status code.

For example:

200 OK → Request successful
400 Bad Request → Request/data is invalid
401 Unauthorized → Authentication problem
403 Forbidden → No permission
404 Not Found → Resource not found
500 Internal Server Error → Server-side problem

So:

Status = What happened?
Data = What is the result?

Complete healthcare example

Imagine this process:

Billing User
↓
RCM System
↓
POST /eligibility
↓
Patient + Insurance Data
↓
Eligibility API
↓
Payer System
↓
Response: Active
↓
200 OK
↓
RCM displays “Insurance Active”

🎯 BA interview perspective

If an interviewer asks you to explain an API request/response, remember:

Endpoint = Where?
Method = What action?
Request Data = What am I sending?
API = Processes the request
Status = Did it work?
Response Data = What did I get back?

---

116. What is Postman?

Postman is a tool commonly used to work with and test APIs. It allows us to send API requests and inspect responses.

You can use it to test:

- GET
- POST
- PUT
- PATCH
- DELETE

and inspect:

- Status code
- Response body
- Headers
- Response time
- Authentication

---

117. How do you test an API using Postman?

Interview-ready flow:

First, I identify the API endpoint and HTTP method.

Then I configure authentication, headers and request parameters or body.

I send the request and verify the HTTP status code, response body, response structure and business data.

I test positive, negative and boundary scenarios.

For example, for an eligibility API, I might test a valid insurance ID, invalid insurance ID, missing required field and unauthorized request.

I compare the actual response against the expected business behavior and document any discrepancies.

---

120. Synchronous vs Asynchronous Communication

Synchronous communication means the sender sends a request and waits for the response before continuing. It is useful when an immediate response is required.

For example, in a healthcare RCM system, when a billing user requests insurance eligibility verification, the RCM system may call the payer API and wait for the eligibility response. Once the response is received, the system displays Active or Inactive status.

Asynchronous communication means the sender sends a request but does not wait for the response immediately. The system can continue with other work, and the response or result is handled later.

For example, if an RCM system sends a large batch of claims to a payer, it may submit the claims and continue processing. The payer can send the results later through a callback, webhook, message queue, or another mechanism.

As a BA, I need to understand which approach the business process requires, especially around response time, dependencies, error handling, retries, notifications, and user experience.

Synchronous

System waits for the response.

Request

↓

Wait

↓

Response

Example:

RCM asks eligibility API → waits for eligibility response.

Asynchronous

System does not need to wait for an immediate response.

Request

↓

Continue

↓

Response/Event later

Example:

Claim submitted → payer processes it → later sends claim-status update.

Memory

Synchronous = Wait

Asynchronous = Continue

📌 SECTION E — HEALTHCARE BA

These are particularly important for you because your **pharmaceutical background + healthcare BA target** can be a strong differentiator.

---

121. What is EHR / EMR?

EMR

Electronic Medical Record

Digital version of a patient's medical record within a healthcare provider organization.

EHR

Electronic Health Record

A broader electronic health record designed to support sharing/coordination of patient health information across authorized healthcare settings.

Interview answer

EMR is generally focused on a patient's digital medical record within a healthcare organization, while EHR is broader and is designed to support information sharing and continuity of care across healthcare settings.

---

122. What is HL7?

HL7 = Health Level Seven

HL7 refers to standards used for exchanging and integrating healthcare information between different systems.

Example:

Hospital System

↓

HL7 Message

↓

Laboratory System

Healthcare systems may exchange information such as:

- Patient information
- Admissions
- Orders
- Results
- Discharge information

---

123. What is FHIR?

FHIR = Fast Healthcare Interoperability Resources

FHIR is a healthcare interoperability standard from HL7 that defines resources and APIs for exchanging healthcare information.

Examples of FHIR resources include:

- Patient
- Observation
- Medication
- Encounter
- Condition

Important distinction

HL7 = broader family of healthcare interoperability standards

FHIR = a modern HL7 standard based around resources and commonly RESTful APIs

---

124. What is HIPAA?

HIPAA = Health Insurance Portability and Accountability Act

For an interview:

HIPAA is a U.S. federal law that includes requirements related to protecting the privacy and security of protected health information and regulating certain healthcare transactions.

BA relevance

When gathering requirements, I need to consider:

- Privacy
- Security
- Access control
- Auditability
- Data protection
- Appropriate disclosure

---

125. What is PHI?

PHI = Protected Health Information

PHI is individually identifiable health information that is protected under applicable healthcare privacy regulations such as HIPAA when handled by covered entities and business associates.

Examples can include:

- Patient name
- Medical information
- Diagnosis
- Treatment information
- Insurance information
- Patient identifiers

BA relevance

Requirements should consider:

Who can access the data?

What data can they see?

How is it protected?

Is access logged?

---

126. What is ICD?

ICD = International Classification of Diseases

ICD is a standardized classification system used to code diagnoses and other health-related conditions.

Example:

A diagnosis is converted into an appropriate ICD code for clinical, billing, reporting or analytical purposes.

---

127. What is CPT?

CPT = Current Procedural Terminology

CPT codes are standardized codes used primarily in the U.S. to represent medical procedures and services.

Simple distinction:

ICD → Diagnosis/condition

CPT → Procedure/service

---

128. What is Claims Processing?

Claims processing is the sequence of activities through which a healthcare claim is prepared, validated, submitted to a payer, adjudicated and ultimately paid, rejected or denied.

Typical flow:

Patient Service

↓

Charge Capture

↓

Claim Creation

↓

Validation

↓

Submission

↓

Payer Adjudication

↓

Accepted / Rejected / Denied / Paid

↓

Payment Posting

---

129. What is Denial Management?

Denial management is the process of identifying, analyzing, correcting and resolving denied healthcare claims to recover appropriate reimbursement and reduce future denials.

Example:

Claim denied because:

Insurance information incorrect.

AR specialist:

1. Investigates
2. Corrects information
3. Resubmits/appeals as appropriate
4. Tracks outcome

BA opportunity

A BA can analyze denial trends and identify process/system improvements.

---

130. What is Revenue Cycle Management?

RCM = Revenue Cycle Management

RCM is the financial process healthcare organizations use to manage revenue from the point of patient registration/service through claim submission, payment and account resolution.

Simplified:

Patient

↓

Registration

↓

Insurance Verification

↓

Service

↓

Charge Capture

↓

Claim

↓

Payer

↓

Payment

↓

Denial / AR Follow-up

↓

Collection

Very important for your interviews

You should be able to explain:

RCM = clinical service → billing → claim → payment → revenue

---

131. What is Eligibility Verification?

Eligibility verification is the process of checking whether a patient's insurance coverage is active and determining relevant coverage information before or around the time services are provided.

Example:

Patient Insurance ID

↓

Eligibility API

↓

Active?

↓

YES

↓

Coverage information

Why important?

It can help reduce:

- Claim rejections
- Unexpected patient responsibility
- Billing errors
- Manual verification

---

132. What is Prior Authorization?

Prior authorization is a process where a payer requires approval before certain healthcare services, procedures or medications are provided or covered.

Example:

Doctor requests procedure

↓

Authorization required?

↓

Yes

↓

Submit authorization request

↓

Payer review

↓

Approved / Denied

BA perspective

You might gather requirements for:

- Authorization request

- Document submission
- Status tracking
- Approval/denial
- Notifications
- Expiration dates

---

133. What is EDI?

EDI = Electronic Data Interchange

EDI is the structured electronic exchange of business documents between organizations using standardized formats.

In healthcare, EDI is widely used for exchanging transactions between providers, payers and other organizations.

Examples:

837 → Claim

835 → Payment/Remittance

---

134. What is an 837 Transaction?

The 837 is a standard healthcare EDI transaction used to electronically submit healthcare claims from providers or their representatives to payers.

Think:

Provider

↓

837 Claim

↓

Payer

It can contain information related to:

- Patient

- Provider
- Services
- Diagnoses
- Charges
- Insurance

BA perspective

You don't necessarily need to construct an 837 manually.

You should understand:

837 = claim submission

---

135. What is an 835 Transaction?

The 835 is a healthcare EDI transaction used by payers to communicate payment and remittance information back to providers or their representatives.

Think:

Payer

↓

835 Remittance

↓

Provider / RCM System

It can communicate information related to:

- Payments
- Adjustments
- Claim-level outcomes
- Patient responsibility
- Reasons for reductions/denials

Easy memory

837 = Send Claim

835 = Receive Payment/Remittance

🧠 MASTER MEMORY MAP

You don't need to memorize 65 answers separately.

Group them.

Requirements

Elicitation

↓

Analysis

↓

Validation

↓

Prioritization

↓

Documentation

↓

Traceability

↓

Change Management

Agile

Product Goal

↓

Product Backlog

↓

Sprint Planning

↓

Sprint Goal

↓

Sprint Backlog

↓

Development

↓

Sprint Review

↓

Retrospective

Technical BA

Requirement

↓

API

↓

Request

↓

Endpoint

↓

Authentication

↓

Response

↓

Status Code

↓

Validation

Healthcare RCM

Patient

↓

Eligibility

↓

Service

↓

Claim

↓

837

↓

Payer

↓

Adjudication

↓

835

↓

Payment

↓

Denial / AR

BA CORE — 136–147

136. What is the difference between Requirement Elicitation and Requirement Gathering?

Requirement elicitation is the broader process of discovering and understanding stakeholder needs, problems, expectations and requirements.

Requirement gathering is often used informally to describe collecting those requirements from stakeholders and other sources.

As a BA, I would say elicitation is more about **discovering the actual need**, while gathering is about **collecting and documenting the information**.

Example

Stakeholder says:

"We need an automated claim system."

I don't simply document that statement. I ask **why**, what problem exists, who is affected and what outcome is expected.

That's **elicitation**.

---

137. Business Requirement vs Functional Requirement

Business Requirement

Describes **what business wants to achieve and why**.

Example:

"Reduce claim-processing time by 30%."

Functional Requirement

Describes **what the system should do** to support that business objective.

Example:

"The system shall automatically validate insurance eligibility before claim submission."

Interview answer

Business requirements focus on the business objective or outcome, whereas functional requirements describe the specific capabilities or behavior the system must provide to achieve that objective.

---

138. Functional vs Non-Functional Requirements

Functional

Describes **what the system should do**.

Example:

"The system shall allow billing staff to submit a claim."

Non-functional

Describes **how well or under what qualities/constraints the system should operate**.

Examples:

"The claim submission response should be returned within 3 seconds."

"Only authorized users should access patient information."

Easy memory

Functional = WHAT

Non-functional = HOW WELL / CONSTRAINT

---

139. What are Business Rules? Give examples.

Business rules are policies, conditions or constraints that define how a business process or system should operate.

Healthcare examples

A claim cannot be submitted if required patient insurance information is missing.

Only authorized billing users can submit claims.

Prior authorization is required for certain procedures based on payer policy.

Important distinction

Requirement: System should validate insurance eligibility.

Business rule: Claim cannot be submitted if insurance coverage is inactive.

---

140. What is Requirement Decomposition?

Requirement decomposition is the process of breaking a large or high-level requirement into smaller, manageable requirements that can be understood, estimated, developed and tested.

Example

High-level requirement:

"Build Claim Management."

Break it into:

Claim Management

↓

Claim Creation

Claim Validation

Claim Submission

Claim Status

Claim Search

Claim Correction

Claim History

In Agile, these may eventually become **features and user stories**.

---

141. How do you identify stakeholders?

I start by understanding the project's objective, business process and affected areas. Then I identify people who are involved in the process, make decisions, provide information, use the system, approve requirements or are impacted by the solution.

For a healthcare RCM project, stakeholders might include:

- Product Owner
- Billing users
- AR specialists
- Provider representatives
- Payer/insurance stakeholders
- Compliance/security teams
- Developers
- QA
- Integration/API teams
- Project Manager

---

142. What is Stakeholder Analysis?

Stakeholder analysis is the process of identifying stakeholders and understanding their roles, interests, influence, expectations and level of involvement in the project.

A common technique is:

Power vs Interest Matrix

High Power + High Interest

↓

Manage Closely

High Power + Low Interest

↓

Keep Satisfied

Low Power + High Interest

↓

Keep Informed

Low Power + Low Interest

↓

Monitor

BA purpose

It helps determine **who needs what level of communication and involvement**.

---

143. What is a Stakeholder Register?

A stakeholder register is a document that records important information about project stakeholders, such as their role, area of interest, influence, expectations, communication needs and involvement.

Example:

Stakeholder	Role	Influence	Interest
Product Owner	Business decision	High	High
Billing Manager	SME	High	High
QA Lead	Testing	Medium	High
Developer	Technical	Medium	Medium

---

144. What is MoM?

MoM = Minutes of Meeting

Minutes of Meeting is a documented summary of a meeting, including key discussions, decisions, action items, owners and deadlines.

Typical format:

Discussion	Decision/Action	Owner	Due Date
Eligibility API	Use payer API	Integration Team	Friday
Claim validation	Add mandatory checks	BA/Dev	Monday

BA responsibility

A BA often prepares or contributes to MoM after important requirement or stakeholder meetings.

---

145. What is a Business Case?

A Business Case explains why a project or initiative should be undertaken. It typically describes the business problem, proposed solution, expected benefits, costs, risks, alternatives and overall justification for investment.

Example:

Problem:

Manual claim verification causes delays.

Proposed solution:

Automated eligibility verification.

Benefits:

- Reduce manual work
- Reduce claim errors
- Improve processing time

Cost:

Development + integration + maintenance.

The Business Case answers:

"Why should the organization invest in this?"

---

146. What is a Project Charter?

A Project Charter is a high-level document that formally authorizes a project and establishes its purpose, objectives, high-level scope, stakeholders, assumptions, constraints, timeline and key responsibilities.

Easy distinction

Business Case → Why should we do it?

Project Charter → What project are we authorizing and at a high level, what will it achieve?

---

147. What is RACI?

RACI = Responsible, Accountable, Consulted, Informed

It defines roles and responsibilities.

Role	Meaning
R	Responsible — does the work
A	Accountable — ultimately owns the outcome
C	Consulted — provides input
I	Informed — kept updated

Example

For approving a claim workflow:

- BA → Responsible
- Product Owner → Accountable
- Billing SME → Consulted
- QA → Informed

Important

There should generally be **one Accountable person/role** for a particular activity or decision.

📌 AGILE — 148–153

148. BA vs Product Owner — What's the difference?

The Product Owner is accountable for maximizing product value and ordering the Product Backlog. The BA focuses more on understanding, analyzing and documenting business needs and translating them into detailed requirements that the delivery team can understand.

A BA can support the Product Owner with requirement analysis, stakeholder discussions, acceptance criteria and backlog refinement.

Simple memory

PO → Product value + backlog ordering

BA → Requirement analysis + detail + clarity

---

149. BA vs Product Manager — What's the difference?

A Product Manager generally focuses on the product's overall strategy, market, customers, roadmap and business outcomes.

A BA generally focuses more on detailed business needs, requirements, processes, stakeholder analysis and translating business needs into implementable requirements.

However, responsibilities vary between organizations, and in some companies the roles overlap.

Simple:

Product Manager → What product should we build and why?

BA → What exactly does the business need and how should the process work?

---

150. Who writes User Stories?

It depends on the organization's process. A Product Owner is accountable for the Product Backlog, while a BA, Product Owner or other team member may write the actual user stories.

In many Agile teams, the BA drafts detailed user stories and acceptance criteria, while the Product Owner validates them from a business-value and priority perspective.

This is a much better answer than saying:

"Only the BA writes user stories."

---

151. Who creates Acceptance Criteria?

Acceptance criteria can be drafted by the BA, Product Owner or collaboratively by the team, depending on the organization's process.

As a BA, I would typically help define clear and testable acceptance criteria based on the business requirement and expected user behavior, and then confirm them with the Product Owner and relevant stakeholders.

---

152. Can a BA change a User Story during a Sprint?

A BA should not independently change the scope or acceptance criteria of a committed story without alignment.

If clarification is needed, I would discuss it with the Product Owner and development and QA teams. If the change materially affects scope or effort, we should assess the impact and follow the team's change process.

The key is to avoid silently changing the agreed requirement during the Sprint.

---

153. What happens if a Story cannot be completed during the Sprint?

First, the team identifies the reason it cannot be completed.

If the story does not meet the Definition of Done by the end of the Sprint, it is generally not considered Done. The team and Product Owner then decide what should happen with the remaining work, such as returning the item to the Product Backlog and re-planning it for a future Sprint.

I would also help identify whether the reason was unclear requirements, dependency, technical complexity or incorrect estimation so we can prevent recurrence.

📌 DOCUMENTATION — 154–159

154. BRD vs SRS vs FRD

This is an important interview question.

Document	Focus
BRD	Business needs
FRD	Functional behavior
SRS	Comprehensive system/software requirements

BRD

Why does the business need this?

FRD

What functions should the system perform?

SRS

What are the complete software/system requirements, including functional and non-functional requirements and relevant interfaces/constraints?

Example

BRD:

Reduce claim processing time.

FRD:

System shall validate claim information before submission.

SRS:

Could contain:

- Functional requirements
- Non-functional requirements
- Interfaces
- Data requirements
- External system interactions
- Constraints

Important interview caveat

Documentation terminology varies between organizations. Some companies use SRS as the main requirements document and may not maintain separate BRD and FRD documents.

That answer shows maturity.

---

155. User Story vs Acceptance Criteria

User Story

Describes the **user need and business value**.

As a billing executive, I want to verify insurance eligibility so that I can reduce claim-related errors.

Acceptance Criteria

Defines **conditions that must be satisfied for the story to be accepted**.

Example:

- Given a valid insurance ID
- When eligibility is requested
- Then the system should display the coverage status.

Memory

User Story = What user needs

Acceptance Criteria = How we know it is acceptable

---

156. Use Case vs User Story

A user story is a concise expression of a user need and business value, commonly used in Agile.

A use case provides more detailed interaction between an actor and the system, including preconditions, main flow, alternative flows and exceptions.

Example:

User Story:

As a billing executive, I want to submit a claim electronically so that I can avoid manual submission.

Use Case:

Actor: Billing Executive

1. Login

2. Search patient

3. Select claim

4. Validate claim

5. Submit claim

6. System generates confirmation

Alternative:

Claim validation fails

→ System displays errors

→ User corrects claim

---

157. Use Case vs Activity Diagram

A use case describes the interaction between an actor and the system to achieve a goal.

An activity diagram visually represents the workflow or sequence of activities, decisions and paths in a process.

Example

Use Case:

Submit Claim

Activity Diagram:

Start

↓

Enter Claim

↓

Validate

↓

Valid?

↙ ↘

No Yes

↓ ↓

Correct Submit

↓

End

---

158. What is a Process Flow?

A process flow is a visual representation of the sequence of activities, decisions, inputs and outputs involved in completing a business process.

Example:

Patient Registration

↓

Eligibility Check

↓

Service

↓

Claim Creation

↓

Claim Validation

↓

Claim Submission

BA uses it to:

- Understand current state
- Identify bottlenecks
- Find manual activities
- Design future state
- Explain requirements

---

159. What is BPMN?

BPMN = Business Process Model and Notation

BPMN is a standardized graphical notation used to model business processes and workflows.

Common elements:

- Events
- Activities/tasks
- Gateways/decisions
- Sequence flows
- Pools
- Lanes
- Message flows

BA example

You could use BPMN to model:

Eligibility Verification → Claim Submission → Payer Response

📌 **API / SQL — 160–169**

160. How do you analyze an API requirement as a BA?

This is an excellent **Technical BA** question.

I first understand the business purpose of the integration and what systems need to communicate.

Then I identify the API operation, endpoint, request data, response data, authentication, business rules, error scenarios and dependencies.

I also clarify data mapping between the source and target systems and define expected success and failure behavior.

Finally, I make sure the requirements are testable and aligned with security and non-functional requirements.

Example

Eligibility API:

RCM System

↓

Eligibility API

↓

Payer

↓

Eligibility Response

---

161. How do you document API requirements?

I would typically document:

API Name

Purpose

Business Scenario

Endpoint

HTTP Method

Authentication

Request Headers

Request Parameters

Request Body

Response Body

Status Codes

Business Rules

Error Scenarios

Data Mapping

Timeout

Retry/Fallback behavior

Security requirements

Dependencies

BA example

API: Insurance Eligibility

Method: GET/POST depending on API design

Purpose: Verify active coverage.

Success: Return eligibility status.

Failure: Return appropriate error and allow fallback/manual workflow if defined.

---

162. What is API Authentication?

API authentication is the process of verifying the identity or credentials of the client or user making an API request.

Common approaches include:

- API key
- Basic authentication
- Bearer token
- OAuth 2.0

- Mutual TLS in some environments

Remember

Authentication answers:

Who are you?

---

163. What is OAuth 2.0?

OAuth 2.0 is an authorization framework that allows an application to obtain limited access to a protected resource without requiring the application to directly handle the user's password.

For a BA, understand the concept rather than implementation details.

Example:

Application

↓

Authorization Server

↓

Access Token

↓

Protected API

---

164. What is a Bearer Token?

A Bearer Token is a credential included with an API request, commonly in the Authorization header, that allows the bearer of the token to access protected resources according to the token's permissions.

Typical format:

Authorization: Bearer <token>

Simple meaning

"I have a valid access token, so the API can authorize my request."

---

165. How would you troubleshoot an API failure?

My approach would be:

1. Check endpoint

2. Check HTTP method

3. Check authentication

4. Check headers

5. Check request payload

6. Check status code

7. Check response/error message

8. Check data mapping

9. Check logs if available

10. Check dependency/external system

11. Reproduce in Postman if appropriate

12. Document and coordinate resolution

Example

If API returns **401**:

I would first check authentication credentials/token rather than immediately assuming there is a code defect.

---

166. Write SQL using GROUP BY

Suppose:

Employee

---

id

name

department

salary

Question:

Find average salary by department.

SELECT department, AVG(salary) AS avg_salary

FROM Employee

GROUP BY department;

BA understanding

GROUP BY groups records based on a column so aggregate functions such as COUNT, SUM, AVG, MIN, and MAX can be applied to each group.

---

167. Write SQL using JOIN

Suppose:

Patient

---

patient_id

patient_name

Claim

---

claim_id

patient_id

claim_amount

Query:

```
SELECT
   p.patient_name,
   c.claim_id,
```

```
    c.claim_amount
FROM Patient p
JOIN Claim c
    ON p.patient_id = c.patient_id;
```

Meaning

We are connecting Patient and Claim using their common patient_id.

---

168. Find Duplicate Records

Example: Find duplicate email addresses.

```
SELECT email, COUNT(*) AS count
FROM Employee
GROUP BY email
HAVING COUNT(*) > 1;
```

Logic

```
GROUP BY
  ↓
COUNT
  ↓
HAVING > 1
  ↓
Duplicates
```

---

169. Find Employees with Salary Greater Than Average

```
SELECT *
FROM Employee
WHERE salary > (
```

```
    SELECT AVG(salary)
    FROM Employee
);
```

Explanation

The subquery calculates average salary, and the outer query returns employees whose salary is greater than that average.

📌 HEALTHCARE — 170–180

170. HIPAA vs PHI

This is commonly confused.

HIPAA

HIPAA is a U.S. federal law that establishes requirements around areas including privacy and security of protected health information and certain healthcare transactions.

PHI

PHI is Protected Health Information—individually identifiable health information that is protected under HIPAA when it falls within HIPAA's scope.

Easy memory

HIPAA = Regulation/Law

PHI = Protected Health Information

---

171. HL7 vs FHIR

HL7 is a standards organization and also commonly refers to a family of healthcare interoperability standards. FHIR is a specific modern HL7 standard that defines healthcare data resources and mechanisms for exchanging them, commonly through RESTful APIs.

Example

FHIR resources:

Patient

Observation

Condition

Medication

Encounter

Easy interview answer

FHIR is one of the modern HL7 standards focused on interoperable healthcare data exchange using standardized resources and APIs.

---

172. EHR vs EMR

EMR is generally the digital medical record maintained within a healthcare organization, while EHR is broader and is designed to support sharing and coordination of patient information across authorized healthcare settings.

Simple:

EMR → Within organization

EHR → Broader longitudinal/shared record

---

173. ICD vs CPT

ICD

Used primarily to represent diagnoses and health conditions.

CPT

Used primarily to represent medical procedures and services in the U.S.

Easy memory

ICD → What condition?

CPT → What service/procedure?

---

174. 837 vs 835

This should be automatic in your interview.

837

Electronic healthcare claim transaction.

Provider

↓

837

↓

Payer

835

Electronic remittance/payment transaction.

Payer

↓

835

↓

Provider/RCM

Memory

837 = Claim goes out

835 = Payment/remittance comes back

---

175. What is a Clearinghouse?

A healthcare clearinghouse is an intermediary that receives healthcare transactions from providers or other organizations, performs validation and formatting according to applicable standards, and routes transactions to the appropriate payer or trading partner.

Simplified:

Provider / RCM

↓

Clearinghouse

↓

Payer

Why useful?

It can help with:

- Standardization
- Validation
- Routing
- Transaction processing

---

176. What is Claim Adjudication?

Claim adjudication is the payer's process of evaluating a submitted healthcare claim against coverage, eligibility, coding, policy and other applicable rules to determine how the claim should be processed and what amount is payable.

Simplified:

Claim

↓

Payer checks

↓

Eligibility

Coverage

Rules

Codes

Benefits

↓

Decision

↓

Pay / Deny / Adjust

---

177. What is a Clean Claim?

A clean claim is a claim submitted with the required information in a correct and acceptable format, with no known issues that would prevent normal processing.

Examples of information that may need to be correct:

- Patient information
- Insurance information
- Provider information
- Diagnosis codes
- Procedure codes
- Required claim fields

Goal

Clean claim → fewer avoidable delays/rejections/denials

---

178. Denial vs Rejection

This is an **important RCM interview question**.

Rejection

A rejection generally occurs when a claim fails validation or formatting checks and is not accepted for normal adjudication.

Example:

Missing required field or invalid format.

Denial

A denial generally occurs when the payer processes/adjudicates the claim but determines that payment will not be made for some or all of the claim based on applicable rules.

Example:

Service not covered or authorization requirement not met.

Memory

Rejection → Failed before/at acceptance

Denial → Payer adjudicated but won't pay as submitted

---

179. What is Patient Responsibility?

Patient responsibility is the portion of the healthcare cost that the patient is responsible for paying after considering the payer's coverage and applicable contractual adjustments.

It can include things such as:

- Deductible
- Copayment
- Coinsurance
- Other applicable patient amounts

Example

Total Charge = $1,000

Insurance Payment = $700

Adjustment = $200

Patient Responsibility = $100

The exact calculation depends on the payer's adjudication and applicable benefit rules.

---

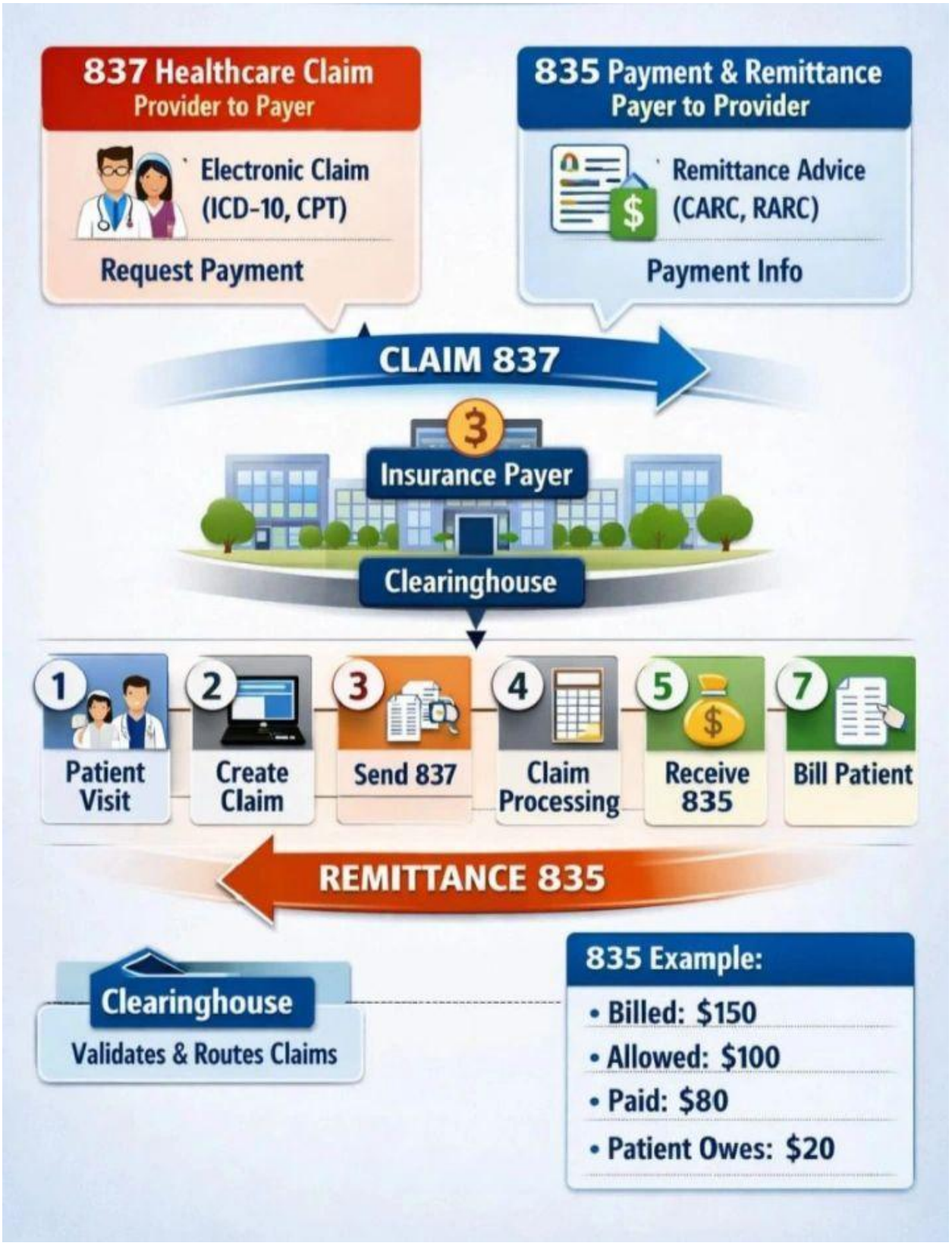
180. Explain the Complete Healthcare RCM Lifecycle

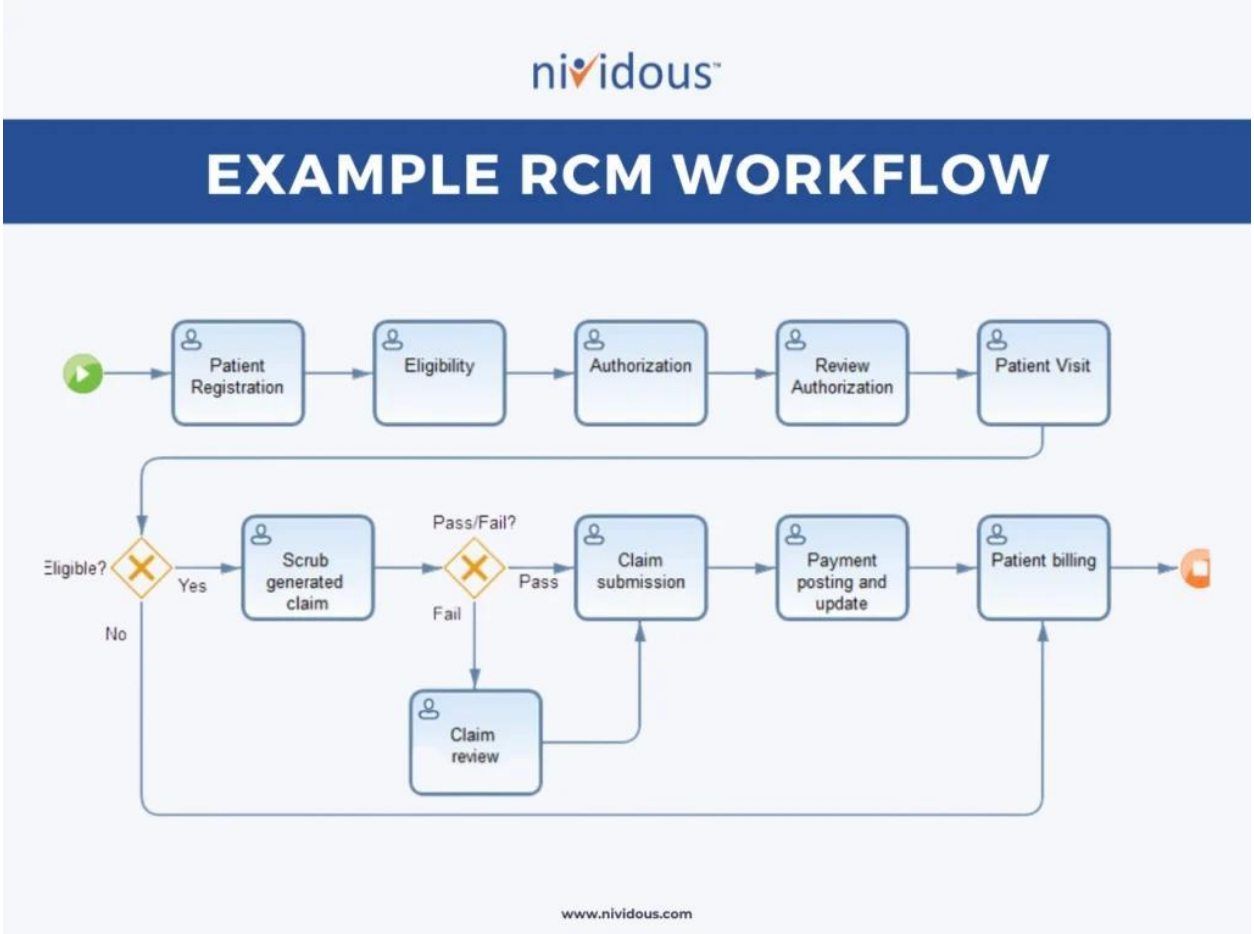
This is one of the **most important questions for your healthcare BA interviews**.

Don't just memorize the individual definitions. Learn this flow:

RCM CYCLE


Start of a Claim
• Patient Registration
• Insurance Eligibility
• Patient Appointment
Claims Submission
• Charge Entry
• Medical Coding
• Timely Claim Submission
Revenue Cycle Management
Claims Management
• Payment Posting
• Medical Coding Services
• Denial Managemet
• Appeals
A/R Management
• A/R Follow up
• Patient Collections
• Patient Statements
Analytics
• Payment Posting
• Medical Coding Services
• Denial Managemet
• Appeals


837 Healthcare Claim
Provider to Payer
Electronic Claim
(ICD-10, CPT)
Request Payment
835 Payment & Remittance
Payer to Provider
Remittance Advice
(CARC, RARC)
Payment Info
CLAIM 837
3
Insurance Payer
Clearinghouse
1
Patient Visit
2
Create Claim
3
Send 837
4
Claim Processing
5
Receive 835
7
Bill Patient
REMITTANCE 835
Clearinghouse
Validates & Routes Claims
835 Example:
• Billed: $150
• Allowed: $100
• Paid: $80
• Patient Owes: $20



6

Interview-ready answer

Revenue Cycle Management is the complete financial lifecycle from patient registration and insurance verification through charge capture, claim submission, payer adjudication, payment posting and accounts receivable or denial management.

A typical lifecycle starts with patient registration and eligibility verification. After healthcare services are provided, charges are captured and the claim is created and validated. The claim may then be submitted electronically, for example using an 837 transaction, often through a clearinghouse. The payer adjudicates the claim and determines payment, adjustment, rejection or denial. Remittance information can come back through an 835 transaction, after which payment is posted and any outstanding accounts receivable or denials are followed up.

Complete flow

1. Patient Registration

↓

2. Insurance Eligibility Verification

↓

3. Prior Authorization (if required)

↓

4. Healthcare Service

↓

5. Charge Capture

↓

6. Medical Coding

↓

7. Claim Creation

↓

8. Claim Validation / Scrubbing

↓

9. Claim Submission

↓

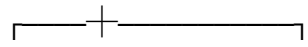
10. Clearinghouse

↓

11. Payer Adjudication

↓

12. Claim Outcome



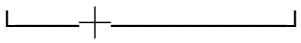
↓ ↓ ↓

Paid Rejected Denied

↓ ↓ ↓

Payment Correction/Appeal

Posting Resubmission



↓

13. Remittance / 835

↓

14. Payment Posting

↓

15. Patient Responsibility

↓

16. AR / Denial Management

↓

17. Account Resolution

As a BA, where do YOU fit?

This is where you should take the answer one step further in an interview:

As a BA, my role can span several parts of this lifecycle. I would understand the current and future-state process, gather requirements from billing and business stakeholders, define business rules, document functional requirements or user stories, support API and EDI integrations, clarify requirements for development and QA, support UAT, and help analyze production issues and process improvements.

That answer connects **healthcare domain + BA responsibilities + technical BA knowledge**.

🎯 ONE IMPORTANT INTERVIEW RULE FOR YOU

Because you're preparing for **Healthcare BA / Technical BA** roles, don't answer technical questions like a developer.

For example, if asked:

"What is an API?"

Don't spend five minutes explaining API architecture.

Say:

"An API is a mechanism that allows two systems to communicate and exchange data. From a BA perspective, I focus on the business purpose of the integration, data being exchanged, request and response, authentication, business rules, error scenarios and expected behavior."

That immediately tells the interviewer:

"This person understands technology from a BA perspective."

🧠 Your 136–180 Final Revision Map

Area	Remember
Elicitation	Discover the real need
Analysis	Refine and understand
Validation	Confirm correctness
Prioritization	Decide importance
Traceability	Requirement → Test
BRD	Business need
FRD	Functional behavior
SRS	Comprehensive system requirements
RACI	Responsibility
MoM	Meeting record
Product Owner	Product value/backlog ordering
BA	Requirements/process/detail
User Story	User need + value
Acceptance Criteria	Conditions for acceptance
BPMN	Business process modeling

Area	Remember
API	System-to-system communication
Endpoint	API access point
OAuth	Authorization framework
Bearer Token	Access credential
837	Claim
835	Remittance
Clearinghouse	Transaction intermediary
Adjudication	Payer decision
Rejection	Not accepted for normal processing
Denial	Processed but payment denied
ICD	Diagnosis
CPT	Procedure/service
EHR	Broader electronic health record
EMR	Organization-focused digital record
HL7	Healthcare interoperability standards
FHIR	Modern HL7 interoperability standard
HIPAA	U.S. privacy/security/transaction law
PHI	Protected health information
RCM	Complete revenue cycle

Next step should be the mock interview. I recommend we don't start by randomly asking technical questions. We'll simulate a **real Healthcare/Technical BA interview** in this order:

Introduction → Experience → Project → Requirements → Agile → SQL → API → Healthcare → Scenario/Behavioral → HR

Other

"What is RAID?"

You can answer:

RAID stands for Risks, Assumptions, Issues, and Dependencies. It is a project management tracking mechanism used to identify and monitor important factors that can affect project delivery. As a BA, I use the RAID log to track risks, validate assumptions, monitor issues, and identify dependencies with other teams or systems.

One important interview distinction

Risk vs Issue is frequently asked:

Risk is a potential future problem; an issue is a problem that has already occurred.

Example:

Risk: API *may* be delayed.
Issue: API *is already delayed*.